

网络管理课程实践指导书

一、研究目的与意义

- 1 掌握基本的网络与系统监控的手段与方法
- 2 掌握基本的拓扑结构监控手段与方法
- 3 掌握 DNS 服务配置的方法

二、实践课程实现方法

- 1 三种完成方法可以选择完成：

- 完成方法 A：标准的网络管理课程实践内容，其中程序采集数据的方法为异步数据采集
- 完成方法 B：较标准的网络管理课程实践内容，其中程序采集数据的方法为同步数据采集
- 完成方法 C：非标准的网络管理课程实践内容，针对编程困难的同学，不需要编写程序，但需要完成企业内部拓扑构建、DNS 服务配置、what'sup 拓扑监控配置（what'sup 程序见压缩包）、solarwinds 数据采集程序配置（solarwinds 程序见压缩包）
其中：可以将 what'supt 程序/solarwinds 程序替换为 Zabbix 或 Nagios 系统（Nagios 系统配置见压缩包）

三、课程实践内容

课程实践内容如下：

- 1 构建最基本的企业内部网络拓扑结构

基本要求：(1)可以使用任何虚拟环境构建企业内部网络拓扑结构；(2)全网连通；(3)本地建立虚拟机，虚拟机内运行 DNS 服务(bind)

- 2 基于 SNMP 协议 libnetsnmp 库编程实现基于同步或异步的数据采集进程

要求：

- 1)采集运行所有运行设备的 9 项运行数据

(1)sysName;(2) ifInOctets;(3)ifInUcastPkts;(4)ifInNucastPkts;(5)ifInErrors;(6)ifOutOctets;(7)ifOutUcastPkts(8)ifOutNucastPkts,ifOutErrors 数值

可以采用同步或异步程序采集数据，所有采集到的数据保存到 DBMS 中，轮询间隔期为 1 分钟，上述采集对象需要由配置文件中读出

- 2) 采集运行 DNS 服务的虚拟机的 6 项指标

(1)物理内存大小(单位 kbytes)；(2)物理内存 used 大小(单位 kbytes)；

(3)物理硬盘总量；(4)交换分区总量；(5) /分区 used 大小

(6)named 进程占用 CPU 时间、占用内存大小

可以采用同步或异步程序采集数据，所有采集到的数据保存到 DBMS 中，轮询间隔期为 1 分钟，上述采集对象需要由配置文件中读出

- 3) 编写 Web 或 QT6 客户端程序显示采集到的数据，客户端程序的要求如下：

- ①后端采用 MVC API（接口）方式编程
- ②前端可以采用 VUE 或 React 或 QT6 编程
- ③也可以不分前端、后端，直接在服务器端渲染 Web 程序
- ④所有数据需要采用 List 方式显示
- ⑤至少要有一组数据采用拆线图或柱状图显示

3 采用 JS(vis.js)编写拓扑结构监控程序，要求如下

- 1) 正常连通（可以使用 ICMP 报文做为判断依据）网络节点为绿色
- 2) 不能连通网络节点为红色
- 3) 测试连通的报文可以是 ICMP 报文，也可以是 telnet 的实现

四、 评分标准与评分矩阵

网络管理课程实践评分标准与评分矩阵

实践内容	完成方法 A	分值	完成方法 B	分值	完成方法 C	分值
企业内部网络构建与 DNS 服务	企业内部网络构建	10	仅企业内部网络构建	10	仅企业内部网络构建	10
	DNS 服务工作正常	10	DNS 服务工作正常	10	DNS 服务工作正常	10
数据采集与显示	异步数据采集	20	同步数据采集	15	Solarwinds 数据采集与图形化显示 注：solarwinds 为基于 SNMP 协议的数据采集与显示客户端程序	20
	数据读取 list 显示	10	数据读取 list 显示	10		
	数据读取图形化显示	10	数据读取图形化显示	10		
拓扑结构监控	JS 编程实现拓扑结构监控	20	Js 编程实现静态拓扑结构	15	采用 what'sup 实现拓扑结构监控 注：what'sup 为拓扑结构监控客户端程序	10
实践报告	按格式完整编写实践报告	20	按格式要求编写实践报告	15	按格式完整撰写实践报告	10
总分		100		85		60

五、 实践完成后答辩

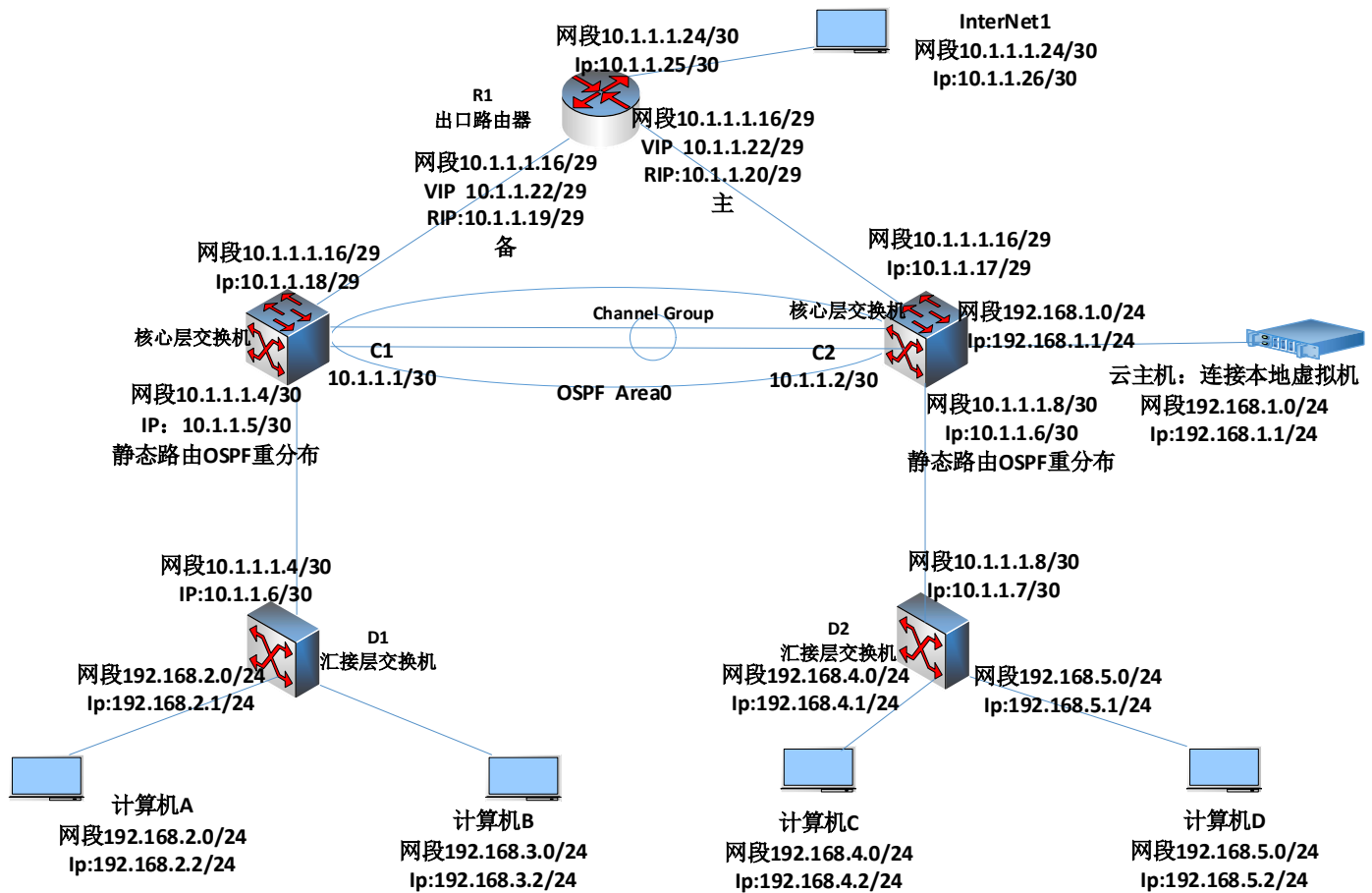
实践完成后，可以随时约老师答辩，答辩时需要

- 1) 完整展示实践内容
- 2) 答辩完成后交实践报告（电子版本，答辩完成后拷给老师）

1 企业内部网络拓扑结构构建

1.1 拓扑结构

网络管理课程实践构建的企业内部网络拓扑结构如下：



1.2 构建技术分析

1.2.1 核心层

由 C1 与 C2 核心层交换机组成核心骨干层

路由协议：OSPF 路由协议

路由协议广播端口：仅 C1 与 C2 连接端口

互联链路：双链路组成 channel-group，负载算法自决定

C1 与 D1/C2 与 D2 路由学习方法：静态路由重分布

1.2.2 WAN 区域

WAN 区域仅由出路 R1 组成

R1 与互联网主机 InetNet1 的连接：R1 默认路由/InetNet1 静态路由（注意用户网络为 192.168.0.0/16 网段）

R1 与核心交换机的连接：VRRP，主/备方式，Vip 为 10.1.1.22，该 VIP 是 C1 与 C2 的默认网关

1.2.3 汇接层

汇接层交换机为 D1 与 D2

与核心层间的路由方法：静态路由重分布（注意用户网络为 192.168.0.0/16 网段）

D1/D2 路由：默认路由

1.2.4 交换区块

存在 D1 交换区块与 D2 交换区块，各包含 2 个 C 类网段

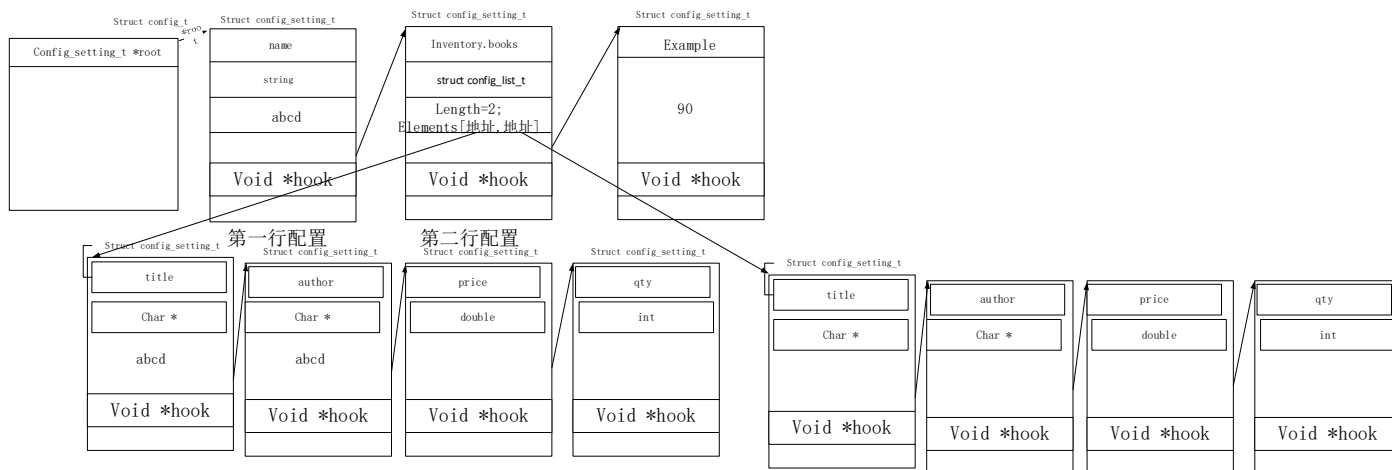
D1 汇接层交换机接入网段：192.168.2.0/24； 192.168.3.0/24

D2 汇接层交换机接入网段：192.168.4.0/24； 192.168.5.0/24

1.2.5 对内服务器

对内服务器连接于 C2 核心交换机，进入 OSPF 路由域的方法，静态路由重分布（直连）

2 配置文件读取（libconfig.so）



2.1 结构体定义----/usr/local/include/libconfig.h

```
#ifndef __libconfig_h
#define __libconfig_h
```

```
#define LIBCONFIG_VER_MAJOR    1
#define LIBCONFIG_VER_MINOR    5
#define LIBCONFIG_VER_REVISION 0
```

```
#include <stdio.h>
```

```
#define CONFIG_TRUE  (1)
#define CONFIG_FALSE (0)
```

```
typedef union config_value_t
{
    int ival;
    long long llval;
    double fval;
    char *sval;
    struct config_list_t *list;
} config_value_t;
```

//值存放在数据结构

//值为 int 类型

//值为 real

//值为 double

//值为 string

//指向 directive 的指针数组的起始位置，用于处理此 directive

//包含有子 directive 的情况

```
typedef struct config_setting_t
{
```

//directive 配置信息

```
    char *name;
```

//名称

```
    short type;
```

//类型

```
    short format;
```

//格式,默认为“=”

```
    config_value_t value;
```

//值 config_value_t 新的数据类型（UNION）

```
    struct config_setting_t *parent;
```

//单向链表

```
    struct config_t *config;
```

//全局信息结构体，因为提高速度，采用冗余的措施

```
    void *hook;
```

//回调函数 key/value key/value

```
    unsigned int line;
```

//行

```
    const char *file;
```

//配置文件名

```
} config_setting_t;
```

//config_setting_t variable_name;

```

typedef enum
{
    CONFIG_ERR_NONE = 0,
    CONFIG_ERR_FILE_IO = 1,
    CONFIG_ERR_PARSE = 2
} config_error_t;

typedef struct config_list_t // directive 中包含有子 directive 时, 子 directive 放入指针数组
{
    unsigned int length;
    config_setting_t **elements; //它是个指针数组, 每个成员都是一个 struct config_setting_t 结构体
} config_list_t;
    [ 地址      地址      地址]
    key5/value5    key6/value6    key7/value7]
    config_setting_t    config_setting_t    config_setting_t

typedef struct config_t //全局信息      结构体    Config_t
{
    config_setting_t *root; //directive 单向链表的起始节点,单向链表的数据类型: config_setting_t
    //typedef 定义的新的数据类型, 单向链表的每个节点是一个结构
    void (*destructor)(void *); //简单指针函数
    int options;
    unsigned short tab_width; //定义分隔符的宽度    key value /key=value
    short default_format; //默认格式    key    value    (间隔符是<tab>)
    const char *include_dir; //包含的目录, 配置文件中目录
    const char *error_text;
    const char *error_file;
    int error_line;
    config_error_t error_type;
    const char **filenames; //打开的文件名指针数组, 可以读取多个配置文件
    unsigned int num_filenames; //打开的文件数量
} config_t;

Key1=value1
Key2=value2
Key3=value3
Key4=(key5=value5,key6=value6)

```

2.2 函数

1) void config_init(config_t * config);

在内存中生成 config_t 结构体并初始化, 将指向该结构体的指针保存于*config

void config_destroy(config_t * config); deallocating 所有涉及此配置文件读取中使用的内存

2) int config_read_file(config_t * config, const char * filename);

作用: 读取并解析由*filename 指定的配置文件, 并将解析的结构写入 config_t 结构体 (*config)

返回值: CONFIG_TRUE on success

CONFIG_FALSE on failure,config_error_text(); config_error_line()用于获取具体的错误信息

配置文件: key=value

Key value value int,shortint, lang lang,double, *char, bool

3) int config_lookup_string(const config_t * config, const char * path, const char **value);

作用: 此函数在打开的 config_t 结构体中查询由*path 指定配置项目的值, 并将值存储于**value

返回值: on success:将由*path 指定的配置项目的值存储于**value 后, 返回 CONFIG_TURE

on failure:由*path 指定的配置项目没有发现或其值的数据类型不匹配(即不是 string)则返回 CONFIG_FALSE

说明: 存储于**value 中的值由 library 管理, 当撤销 config_t 时自动 released

int config_lookup_int (const config_t * config, const char * path,int * value)

int config_lookup_int64 (const config_t * config, const char * path, long long * value)

int config_lookup_float (const config_t * config, const char * path, double * value)

int config_lookup_bool (const config_t * config, const char * path, int * value)

int config_lookup_string (const config_t * config, const char * path, const char ** value)

4) config_setting_t * config_lookup(const config_t * config, const char * path);

作用: 此函数定位由*path 指定的配置项目在配置文件中的位置

返回值: on success:返回指向 config_setting_t 结构体的指针 (config_setting_t 结构体是一个单向链表)

NULL 由*path 指定的配置项目在配置文件中没有发现

5) 例:

```
int main(int argc, char **argv)
{
    config_t cfg;
    config_setting_t *setting;
    const char *str;

    config_init(&cfg);

    if(! config_read_file(&cfg, "example.cfg"))                /* Read the file. If there is an error, report it and exit. */
    {
        fprintf(stderr, "%s:%d - %s\n", config_error_file(&cfg),
            config_error_line(&cfg), config_error_text(&cfg));
        config_destroy(&cfg);
        return(EXIT_FAILURE);
    }

    if(config_lookup_string(&cfg, "name", &str))                /* Get the store name. */
        printf("Store name: %s\n", str);
    else
        fprintf(stderr, "No 'name' setting in configuration file.\n");

    setting = config_lookup(&cfg, "inventory.books");           /* Output a list of all books in the inventory. */
}
```

6) int config_setting_length(const config_setting_t *setting);

作用: 该函数返回配置文件中一个配置项目组中的成员的个数, 或返回一个配置项目列表或 array 的成员数

例: 配置文件 example.cfg 中存在有两个配置项目组

```
books = ( { title   = "Treasure Island";
            author = "Robert Louis Stevenson";
            price  = 29.99;
            qty    = 5; },
          { title   = "Snow Crash";
            author = "Neal Stephenson";
            price  = 9.99;
            qty    = 8; }
        );
```

程序:

```
setting = config_lookup(&cfg, "inventory.books");
if(setting != NULL)
{
    int count = config_setting_length(setting);
    int i;

    for(i = 0; i < count; ++i)
    {
        config_setting_t *book = config_setting_get_elem(setting, i);

                                                                    /* Only output the record if all of the expected fields are present. */

        const char *title, *author;
        double price;
        int qty;

        if(!(config_setting_lookup_string(book, "title", &title)
            && config_setting_lookup_string(book, "author", &author)
            && config_setting_lookup_float(book, "price", &price)
            && config_setting_lookup_int(book, "qty", &qty)))
            continue;
    }
}
```

7). config_setting_t * config_setting_get_elem(const config_setting_t *setting, unsigned int index);

作用: 此函数取出以 index 方式 (i=0、1、2...) 指定的成员, 该成员必须是 group, array, list 中的成员

返回值: on success, 返回索引的成员

NULL, 如果 index 超出了成员个数的范围或 setting 不是 array, list, 或 group

8).int config_setting_lookup_string(const config_setting_t *setting, const char *name, const char **value);

作用: 此函数查询以*name 指定的子配置项目的配置值(字符串类型), 并将值存储于**value 中

返回值: on success, 将值存储于**value 中并返回 CONFIG_TRUE

CONFIG_FALSE, 如果以*name 指定的子配置项目没有发现或数据类型不匹配

说明：存储于**value 中的值由 library 管理，当撤销 config_t 时自动 released

```
int config_setting_lookup_int(const config_setting_t *setting, const char *name, const char **value);
```

作用：与 8 相同，以 *name 指定的子配置项目值的数据类型为 int

```
int config_setting_lookup_int64(const config_setting_t *setting, const char *name, const char **value);
```

作用：与 8 相同，以 *name 指定的子配置项目值的数据类型为 int64

```
int config_setting_lookup_float(const config_setting_t *setting, const char *name, const char **value);
```

作用：与 8 相同，以 *name 指定的子配置项目的值的数据类型为 float

```
int config_setting_lookup_bool(const config_setting_t *setting, const char *name, const char **value);
```

作用：与 8 相同，以 *name 指定的子配置项目的值的数据类型为 bool

```
9).const char * config_error_text(const config_t *config);
```

```
int config_error_line(const config_t * config);
```

作用：这些函数由宏实现，文本和解析错误的行编号。

2.3 编译：

因为#include <libconfig.h>,所以需要在链接时指定 -lconfig 作为链接参数

```
# gcc example1.c -o example1 -lconfig
```

```
或# gcc example1.c -o example1 -I/usr/local/include/ -lconfig
```

2.4 示例源代码：# more example1.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <libconfig.h>
```

```
/* This example reads the configuration file 'example.cfg' and displays some of its contents. */
```

```
int main(int argc, char **argv)
```

```
{
```

```
    config_t cfg;    //cfg 变量 config_t
```

```
    config_setting_t *setting;    //指针，指向一个节点
```

```
    const char *str;
```

```
    config_init(&cfg);    //config_setting_t 的链表起始点在
```

```
    //cfg 所占内存区载的结构体中
```

```
    /* Read the file. If there is an error, report it and exit. */
```

```
    if(! config_read_file(&cfg, "example.cfg"))
```

```
    {
```

```
        fprintf(stderr, "%s:%d - %s\n", config_error_file(&cfg),  
                    config_error_line(&cfg), config_error_text(&cfg));
```

```
        config_destroy(&cfg);
```

```
        return(EXIT_FAILURE);
```

```
    }
```

```
    /* Get the store name. */    key    value
```

```
    if(config_lookup_string(&cfg, "name", &str))    //在单向链表中遍历，以 name 为 key，读取它的值
```

```
        printf("Store name: %s\n\n", str);
```

```
    else
```

```
        fprintf(stderr, "No 'name' setting in configuration file.\n");
```

```
    /* Output a list of all books in the inventory. */
```

```
    setting = config_lookup(&cfg, "inventory.books");    //config_lookup() 读取混合的数据子类型
```

```
    if(setting != NULL)
```

```
    {
```

```
        int count = config_setting_length(setting);
```

```
        int i;
```

```
        printf("%-30s  %-30s  %-6s  %s\n", "TITLE", "AUTHOR", "PRICE", "QTY");
```

```
        for(i = 0; i < count; ++i)
```

```
        {
```

```
            config_setting_t *book = config_setting_get_elem(setting, i);    config_setting_get_elem()读取其中所有的成员
```

```
            //books 就是一个 config_setting_t 的一个节点
```

```
            //该节点填充了该结构体中的 struct config_list_t 值
```

```

/* Only output the record if all of the expected fields are present. */
const char *title, *author;
double price;
int qty;

if(!(config_setting_lookup_string(book, "title", &title)
    && config_setting_lookup_string(book, "author", &author)
    && config_setting_lookup_float(book, "price", &price)
    && config_setting_lookup_int(book, "qty", &qty)))
    continue;

printf("%-30s  %-30s  $%6.2f  %3d\n", title, author, price, qty);
}
putchar('\n');
}

config_destroy(&cfg);
return(EXIT_SUCCESS);
}

```

结果: # ./example1

Store name: Books, Movies & More

TITLE	AUTHOR	PRICE	QTY
Treasure Island	Robert Louis Stevenson	\$ 29.99	5
Snow Crash	Neal Stephenson	\$ 9.99	8

3 基于 libnetsnmp.so 的同步 IO

3.1 snmp_session 结构

```

struct snmp_session {
    long            version;           /* snmp version */
    int             retries;           /* Number of retries before timeout. */
    long           timeout;            /* Number of uS until first timeout, then exponential backoff */
    u_long          flags;
    struct snmp_session *subsession;
    struct snmp_session *next;
    char            *peername;         //对端主机名，需要一种方法，将主机名->IP 地址 char *"211.71.149.251"
    u_short         remote_port;       //udp 161
    char            *localname;        /* My Domain name or dotted IP address, 0 for default */
    u_short         local_port;        //>1024 随机端口
    u_char          *(*authenticator)(u_char *, size_t *, u_char *, size_t); //回调函数
    /* Authentication function or NULL if null authentication is used */
    netsnmp_callback callback;         /* Function to interpret incoming data */
    void            *callback_magic;    /* Pointer to data that the callback function may consider important */
    int             s_errno;            /* copy of system errno */
    int             s_snmp_errno;       /* copy of library errno */
    long           sessid;              /* Session id - AgentX only */
    /* SNMPv1 & SNMPv2c fields */
    u_char          *community;        /* community for outgoing requests. */ u_char 不局限于 ASCII
    size_t          community_len;      /* Length of community name. */
    size_t          rcvMsgMaxSize;      /* Largest message to try to receive. */
    size_t          sndMsgMaxSize;      /* Largest message to try to send. */
}

```

3.2 netsnmp_pdu 结构

```

typedef struct snmp_pdu {
#define non_repeaters    errstat
#define max_repetitions  errindex
    /** Protocol-version independent fields */
    long            version;           /* snmp version */
    int             command;          /* Type of this PDU */
    long           reqid;              /* Request id - note: not incremented on retries */
    long           msgid;              /* Message id for V3 messages note: incremented for each retry */
    long           transid;            /* Unique ID for incoming transactions */
    long           sessid;             /* Session id for AgentX messages */
    long           errstat;            /* Error status (non_repeaters in GetBulk) */
    long           errindex;           /* Error index (max_repetitions in GetBulk) */
    u_long         time;              /* Uptime */
}

```

```

    u_long      flags;
    int         securityModel;
    int         securityLevel;          /** noAuthNoPriv, authNoPriv, authPriv */
    int         msgParseModel;
    /* Transport-specific opaque data. This replaces the IP-centric address * field. */
    void        *transport_data;
    int         transport_data_length;
    /* The actual transport domain. This SHOULD NOT BE FREE()D. */
    const oid   *tDomain;
    size_t      tDomainLen;
    netsnmp_variable_list *variables;

    u_char      *community; /** community for outgoing requests. */
    size_t      community_len; /** length of community name. */
} netsnmp_pdu;

```

3.3.netsnmp_variable_list 结构

```

typedef struct variable_list {
    struct variable_list *next_variable;          /** NULL for last variable */
    oid                  *name;                   /** Object identifier of variable */
    size_t               name_length;             /** number of subid's in name */
    u_char               type;                   /** ASN type of variable */
    netsnmp_vardata val;                         /** value of variable */
    size_t               val_len;                /** the length of the value to be copied into buf */
    oid                  name_loc[MAX_OID_LEN];  /** buffer to hold the OID */
    u_char               buf[40];                /** 90 percentile < 40. */
    void                 *data;                  /** (Opaque) hook for additional data */
    void                 (*dataFreeHook)(void *); /** callback to free above */
    int                  index;
} netsnmp_variable_list;

```

3.4 Netsnmp_vardata 结构

```

typedef union {
    long          *integer;
    u_char        *string;
    oid           *objid;
    u_char        *bitstring;
    struct counter64 *counter64;
    #ifdef NETSNMP_WITH_OPAQUE_SPECIAL_TYPES
        float      *floatVal;
        double     *doubleVal;
    #endif
} netsnmp_vardata;

```

3.5 完整源代码

```

#include <net-snmp/net-snmp-config.h>
#include <net-snmp/net-snmp-includes.h>
#include <string.h>

int main(int argc, char ** argv)
{
    netsnmp_session session, *ss;
    netsnmp_pdu *pdu;
    netsnmp_pdu *response;

    oid anOID[MAX_OID_LEN];
    size_t anOID_len;

    netsnmp_variable_list *vars;
    int status;
    int count=1;

    init_snmp("snmpdemoapp");

    snmp_sess_init( &session );
    session.peername = strdup("211.71.149.251");

    session.version = SNMP_VERSION_1;

    session.community = "bjfu18";
    session.community_len = strlen(session.community);
    ss = snmp_open(&session);
    if (!ss) {

        /** Initialize the SNMP library */
        /** Initialize a "session" that defines who we're going to talk to */
        /** set up defaults */
        /** malloc()申请内存, 将字符串拷入, 返回指向 memory 的指针 */
        /** set the SNMP version number */
        /**SNMP_VERSION_2c */
        /** set the SNMPv1 community name used for authentication */

        /** establish the session */
    }
}

```

```

    snmp_sess_perror("ack", &session);
    exit(1);
}
/* Create the PDU for the data for our request. 1) We're going to GET the system.sysDescr.0 node.*/
pdu = snmp_pdu_create(SNMP_MSG_GET);
anOID_len = MAX_OID_LEN;
if (!snmp_parse_oid(".1.3.6.1.2.1.1.1.0", anOID, &anOID_len)) {
    snmp_perror(".1.3.6.1.2.1.1.1.0");
    exit(1);
}
#ifdef OTHER_METHODS
/* These are alternatives to the 'snmp_parse_oid' call above, e.g. specifying the OID by name rather than numerically.*/
read_objid(".1.3.6.1.2.1.1.1.0", anOID, &anOID_len);
get_node("sysDescr.0", anOID, &anOID_len);
read_objid("system.sysDescr.0", anOID, &anOID_len);
#endif

snmp_add_null_var(pdu, anOID, anOID_len);
status = snmp_synch_response(ss, pdu, &response); /* Send the Request out.*/

if (status == STAT_SUCCESS && response->errstat == SNMP_ERR_NOERROR) { /* Process the response.*/
    /* SUCCESS: Print the result variables*/
    for(vars = response->variables; vars; vars = vars->next_variable)
        print_variable(vars->name, vars->name_length, vars);
    /* manipulate the information ourselves */
    for(vars = response->variables; vars; vars = vars->next_variable) {
        if (vars->type == ASN_OCTET_STR) {
            char *sp = (char *)malloc(1 + vars->val_len);
            memcpy(sp, vars->val.string, vars->val_len);
            sp[vars->val_len] = '\0'; /*字符串的结束符*/
            printf("value #%d is a string: %s\n", count++, sp);
            free(sp);
        }
        else
            printf("value #%d is NOT a string! Ack!\n", count++);
    }
} else {
    /* FAILURE: print what went wrong!*/
    if (status == STAT_SUCCESS)
        fprintf(stderr, "Error in packet\nReason: %s\n", snmp_errstring(response->errstat));
    else if (status == STAT_TIMEOUT)
        fprintf(stderr, "Timeout: No response from %s.\n", session.peername);
    else
        snmp_sess_perror("snmpdemoapp", ss);
}

```

3.6 结果与输出

4 数据库连接

4.1 连接例程

用 C 语言连接 MySQL 数据库包含两个步骤

1) 初始化一个连接句柄结构

```
#include "mysql.h"
```

```
MYSQL *mysql_init(MYSQL *);
```

作用：初始化连接句柄

返回值：通常传递 NULL 给这个例程，它会返回一个指向新分配的连接句柄结构的指针
若传递一个已有的结构，它将被重新初始化。

出错时返回 NULL

2) 实际进行连接

```
#include "mysql.h"
```

```
MYSQL *mysql_real_connect(MYSQL *connection,
```

```

const char *server_host,
const char *sql_user_name,
const char *sql_password,
const char *db_name,
unsigned int port_number,
const char *unix_socket_name,
unsigned int flags);

```

作用：进行实际连接

参数：connection 必须指向已经被 mysql_init 初始化过的结构

Sql_user_name 和 sql_password 如果登录名为 NULL，则假设登录名为当前 Linux 用户的登录 ID，
如果密码为 NULL，将只能访问服务器上无需密码就可访问的数据，密码会在
通过网络传输前进行加密

Port_number 和 unix_socket_name 应该分别为 0 和 NULL

flags 用来对一些定义的位模式进行 OR 操作

返回值：若无法连接，它将返回 NULL，mysql_error 函数可以提供有帮助的信息

3) 关闭连接

```
#include "mysql.h"
```

```
Void mysql_close(MYSQL *connection);
```

如果连接是由 mysql_init 建立的，MYSQL 结构会被释放。指针将会失效并无法再次使用

4)、mysql_options 例程

可以在 mysql_init 和 mysql_real_connect 之间调用设置一些选项

```
#include "mysql.h"
```

```
Int mysql_options(MYSQL *connection, enum option_to_set, const char *argument);
```

说明：mysql_options 例程一次只能设置一个选项，所以每设置一个选项就得调用它一次，同时，该例程必须出现在 mysql_init 和 mysql_real_connect 之间即可。

选 项

enum 选项	实际参数类型	说明
MYSQL_OPT_CONNECT_TIMEOUT	Const unsigned int *	连接超时之前的等待秒数
MYSQL_OPT_COMPRESS	None, 使用 NULL	网络连接中使用压缩机制
MYSQL_INIT_COMMAND	Const char *	每次连接建立后发送的命令

返回值：一次成功的调用将返回 0。因为它仅仅是用来设置标志，所以失败总是意味着使用了一个无效的选项。

例：如果要设置连接超时时间为 7 秒，其代码片断如下：

```
Unsigned int timeout = 7;
```

```
...
```

```
connection=mysql_init(NULL);
```

```
ret=mysql_option(connection, MYSQL_OPT_CONNECTION_TIMEOUT, (const char *)&timeout);
```

```
if (ret){
```

```
    /Handle error */
```

```
    ...
```

```
}
```

```
connection=mysql_real_connect(connection ...
```

4.2 错误处理

MySQL 使用由连接句柄结构报告的返回码，两个必备的例程是：

```
Unsigned int mysql_errno(MYSQL *connection); 返回错误码
```

和

```
char *mysql_error(MYSQL *connection); 返回错误码对应的文本信息
```

可以通过调用 mysql_errno 并传递连接结构来获得错误码（错误码在头文件 errmsg.h（报告客户端错误）或 mysqld_error.h（报告服务器端错误）中定义），错误码通常都是非 0 值。如果未设定错误码，它将返回 0。
每次调用库都会更新错误码，所以只能得到最后一个执行命令的错误码。

4.3 执行 SQL 语句

1) 执行 SQL 语句的主要 API 函数

```
Int mysql_query(MYSQL *connection, const char *query);
```

说明：此例程接受连接结构指针和文本字符串形式的有效 SQL 语句

返回值：如果成功，返回 0

2) 检查受影响的行数

My_ulonglong mysql_affected_rows(MYSQL *connection);

说明：返回值的数据类型是 my_ulonglong（无符号长整型），使用 printf 时，应使用的转换符为 %lu

作用：函数返回受 update, insert 或 delete 查询影响的行数

返回值：0 没有受影响的行数

正数 受语句影响的行数

4.4 例：更新数据---update1.c

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  #include "mysql.h"
5
6  int main(int argc, char *argv[]) {
7      MYSQL my_connection;
8      int res;
9
10     mysql_init(&my_connection);
11     if (mysql_real_connect(&my_connection, "localhost",
12                           "username", "secret", "foo", 0, NULL, 0)) {
13         printf("Connection success\n");
15         res = mysql_query(&my_connection, "UPDATE children SET AGE = 4 WHERE fname = 'Ann'");
16         if (!res) {
17             printf("Updated %lu rows\n", (unsigned long)mysql_affected_rows(&my_connection));
18         } else {
19             fprintf(stderr, "Update error %d: %s\n", mysql_errno(&my_connection),
20                     mysql_error(&my_connection));
21         }
22
23         mysql_close(&my_connection);
24     } else {
25         fprintf(stderr, "Connection failed\n");
26         if (mysql_errno(&my_connection)) {
27             fprintf(stderr, "Connection error %d: %s\n", mysql_errno(&my_connection),
28                     mysql_error(&my_connection));
29         }
30     }
31
32     return EXIT_SUCCESS;
33 }
```

4.5 编译：# gcc -I/usr/include/mysql program.c -L/usr/lib/mysql -lmysqlclient -o program

其中：-I(大写的 i) 指定 include 路径

-L(大写的 L) 指定库文件路径

-l(小写的 l) 指定链接的库模块

5 异步数据采集

5.1 snmp_session 结构

#include <net-snmp/net-snmp-config.h>

#include <net-snmp/net-snmp-includes.h>

```
struct snmp_session {
    long    version; /** snmp version */
    int     retries; /** Number of retries before timeout. */
    long    timeout; /** Number of uS until first timeout, then exponential backoff */
    u_long  flags;
    struct snmp_session *subsession;
    struct snmp_session *next;
    char    *peername;
    u_short remote_port;
    char    *localname; /** My Domain name or dotted IP address, 0 for default */
    u_short local_port;
    u_char  *(*authenticator) (u_char *, size_t *, u_char *, size_t);
}
```

```

/**      * Authentication function or NULL if null authentication is used      */
netsnmp_callback callback;      /* Function to interpret incoming data */

void      *callback_magic;      /* Pointer to data that the callback function may consider important */
int      s_errno;      /* copy of system errno */
int      s_snmp_errno;      /* copy of library errno */
long      sessid;      /* Session id - AgentX only */

/*      * SNMPv1 & SNMPv2c fields      */
u_char      *community; /* community for outgoing requests. */
size_t      community_len; /* Length of community name. */
size_t      rcvMsgMaxSize; /* Largest message to try to receive. */
size_t      sndMsgMaxSize; /* Largest message to try to send. */

/*      * SNMPv3 fields      */
u_char      isAuthoritative; /* are we the authoritative engine? */
u_char      contextEngineID; /* authoritative snmpEngineID */
size_t      contextEngineIDLen; /* Length of contextEngineID */
u_int      engineBoots; /* initial engineBoots for remote engine */
u_int      engineTime; /* initial engineTime for remote engine */
char      *contextName; /* authoritative contextName */
size_t      contextNameLen; /* Length of contextName */
u_char      *securityEngineID; /* authoritative snmpEngineID */
size_t      securityEngineIDLen; /* Length of contextEngineID */
char      *securityName; /* on behalf of this principal */
size_t      securityNameLen; /* Length of securityName. */
oid      *securityAuthProto; /* auth protocol oid */
size_t      securityAuthProtoLen; /* Length of auth protocol oid */
u_char      securityAuthKey[USM_AUTH_KU_LEN]; /* Ku for auth protocol XXX */
size_t      securityAuthKeyLen; /* Length of Ku for auth protocol */
u_char      *securityAuthLocalKey; /* Kul for auth protocol */
size_t      securityAuthLocalKeyLen; /* Length of Kul for auth protocol XXX */
oid      *securityPrivProto; /* priv protocol oid */
size_t      securityPrivProtoLen; /* Length of priv protocol oid */
u_char      securityPrivKey[USM_PRIV_KU_LEN]; /* Ku for privacy protocol XXX */
size_t      securityPrivKeyLen; /* Length of Ku for priv protocol */
u_char      *securityPrivLocalKey; /* Kul for priv protocol */
size_t      securityPrivLocalKeyLen; /* Length of Kul for priv protocol XXX */
int      securityModel; /* snmp security model, v1, v2c, usm */
int      securityLevel; /* noAuthNoPriv, authNoPriv, authPriv */
char      *paramName; /* target param name */
void      *securityInfo; /* security module specific */
struct netsnmp_container_s *transport_configuration; /* transport specific configuration */
void      *myvoid; /* use as you want data */
};

```

5.2 netsnmp_pdu 结构

```

typedef struct snmp_pdu {
#define non_repeaters    errstat
#define max_repetitions errindex
    /* Protocol-version independent fields */
    long      version; /* snmp version */
    int      command; /* Type of this PDU */
    long      reqid; /* Request id - note: not incremented on retries */
    long      msgid; /* Message id for V3 messages note: incremented for each retry */
    long      transid; /* Unique ID for incoming transactions */
    long      sessid; /* Session id for AgentX messages */
    long      errstat; /* Error status (non_repeaters in GetBulk) */
    long      errindex; /* Error index (max_repetitions in GetBulk) */
    u_long      time; /* Uptime */
    u_long      flags;
    int      securityModel;
    int      securityLevel; /* noAuthNoPriv, authNoPriv, authPriv */
    int      msgParseModel;

    /* Transport-specific opaque data. This replaces the IP-centric address field. */
    void      *transport_data;
    int      transport_data_length;

    /* The actual transport domain. This SHOULD NOT BE FREE()D. */
    const oid      *tDomain;
    size_t      tDomainLen;
    netsnmp_variable_list *variables;

    /* * SNMPv1 & SNMPv2c fields */
    u_char      *community; /* community for outgoing requests. */
    size_t      community_len; /* length of community name. */
    /* Trap information */
    oid      *enterprise; /* System OID */
    size_t      enterprise_length;
    long      trap_type; /* trap type */
    long      specific_type; /* specific type */
    unsigned char      agent_addr[4]; /* This is ONLY used for v1 TRAPS */

    /* SNMPv3 fields */
    u_char      *contextEngineID; /* context snmpEngineID */
    size_t      contextEngineIDLen; /* Length of contextEngineID */
    char      *contextName; /* authoritative contextName */
    size_t      contextNameLen; /* Length of contextName */
    u_char      *securityEngineID; /* authoritative snmpEngineID for security */
};

```

```

size_t      securityEngineIDLen; /** Length of securityEngineID */
char        *securityName; /** on behalf of this principal */
size_t      securityNameLen; /** Length of securityName. */
} netsnmp_pdu;

```

5.3 netsnmp_variable_list 结构

```

typedef struct variable_list {
    struct variable_list *next_variable; /* NULL for last variable */
    oid                  *name;          /* Object identifier of variable */
    size_t               name_length;    /* number of subid's in name */
    u_char               type;           /* ASN type of variable */
    netsnmp_vardata      val;            /* value of variable */
    size_t               val_len;        /* the length of the value to be copied into buf */
    oid                  name_loc[MAX_OID_LEN]; /* buffer to hold the OID */
    u_char               buf[40];        /* 90 percentile < 40. */
    void                 *data;          /* (Opaque) hook for additional data */
    void                 (*dataFreeHook)(void *); /* callback to free above */
    int                  index;
} netsnmp_variable_list;

```

5.4 Netsnmp_vardata 结构

```

typedef union {
    long            *integer;
    u_char          *string;
    oid             *objid;
    u_char          *bitstring;
    struct counter64 *counter64;
    #ifdef NETSNMP_WITH_OPAQUE_SPECIAL_TYPES
        float        *floatVal;
        double       *doubleVal;
        /** t_union *unionVal; */
    #endif
} netsnmp_vardata; /* NETSNMP_WITH_OPAQUE_SPECIAL_TYPES */

```

5.5. asynchapp.c 源代码

```

#include <net-snmp/net-snmp-config.h>
#include <net-snmp/net-snmp-includes.h>

```

```

struct host {
    const char *name; /* a list of hosts to query */
    const char *community;
} hosts[] = {
    { "test1", "public" },
    { "test2", "public" },
    { "test3", "public" },
    { "test4", "public" },
    { NULL }
};

```

```

struct oid { /* a list of variables to query for */
    const char *Name;
    oid Oid[MAX_OID_LEN];
    int OidLen;
} oids[] = {
    { "system.sysDescr.0" },
    { "interfaces.ifNumber.1" },
    { "interfaces.ifNumber.0" },
    { NULL }
};

```

-----void initialize(void):处理 struct oid 中的 Oid[MAC_OID_LEN]与 OidLen-----

```

void initialize (void)
{
    struct oid *op = oids;
    init_snmp("asynchapp"); /* initialize library */

    while (op->Name) { /* parse the oids */
        op->OidLen = sizeof(op->Oid)/sizeof(op->Oid[0]);
        if (!read_objid(op->Name, op->Oid, &op->OidLen)) {
            snmp_perror("read_objid");
            exit(1);
        }
        op++;
    }
}

```

-----int print_result()-----

```

int print_result (int status, struct snmp_session *sp, struct snmp_pdu *pdu)
{
    char buf[1024];
    struct variable_list *vp;
    int ix;
    struct timeval now;
    struct timezone tz;

```

```

struct tm *tm;

gettimeofday(&now, &tz);
tm = localtime(&now.tv_sec);
fprintf(stdout, "%.2d:%.2d:%.2d.%6d ", tm->tm_hour, tm->tm_min, tm->tm_sec,
        now.tv_usec);
switch (status) {
case STAT_SUCCESS:
    vp = pdu->variables;
    if (pdu->errstat == SNMP_ERR_NOERROR) {
        while (vp) {
            snprint_variable(buf, sizeof(buf), vp->name, vp->name_length, vp);
            fprintf(stdout, "%s: %s\n", sp->peername, buf);
            vp = vp->next_variable;
        }
    }
    else {
        for (ix = 1; vp && ix != pdu->errindex; vp = vp->next_variable, ix++)
            ;
        if (vp) snprint_objid(buf, sizeof(buf), vp->name, vp->name_length);
        else strcpy(buf, "(none)");
        fprintf(stdout, "%s: %s: %s\n",
            sp->peername, buf, snmp_errstring(pdu->errstat));
    }
    return 1;
case STAT_TIMEOUT:
    fprintf(stdout, "%s: Timeout\n", sp->peername);
    return 0;
case STAT_ERROR:
    snmp_perror(sp->peername);
    return 0;
}
return 0;
}

***** poll all hosts in parallel */*****

struct session {
    struct snmp_session *sess;          /* SNMP session data */
    struct oid *current_oid;           /* How far in our poll are we */
} sessions[sizeof(hosts)/sizeof(hosts[0])];

int active_hosts;                      /* hosts that we have not completed */

int asynch_response(int operation, struct snmp_session *sp, int reqid, struct snmp_pdu *pdu, void *magic) /* response handler */
{
    struct session *host = (struct session *)magic;
    struct snmp_pdu *req;

    if (operation == NETSNMP_CALLBACK_OP_RECEIVED_MESSAGE) {
        if (print_result(STAT_SUCCESS, host->sess, pdu)) {
            host->current_oid++;          /* send next GET (if any) */
            if (host->current_oid->Name) {
                req = snmp_pdu_create(SNMP_MSG_GET);
                snmp_add_null_var(req, host->current_oid->Oid, host->current_oid->OidLen);
                if (snmp_send(host->sess, req))
                    return 1;
            }
            else {
                snmp_perror("snmp_send");
                snmp_free_pdu(req);
            }
        }
    }
    else
        print_result(STAT_TIMEOUT, host->sess, pdu);
    active_hosts--;                      /* something went wrong (or end of variables) * this host not active any more */
    return 1;
}

-----void asynchronous(void)-----
void asynchronous(void)
{
    struct session *hs;
    struct host *hp;

    //给每台主机发送 oids[]中的第一个对象的请求, for 循环发送, 尔后每台主机返回响应后, 再 oids++, 再向该主机发送下一个对象
    //请求数据报 (肯定是在回调函数中完成的)
    for (hs = sessions, hp = hosts; hp->name; hs++, hp++) { /* startup all hosts */
        struct snmp_pdu *req;
        struct snmp_session sess;
        snmp_sess_init(&sess);          /* initialize session */

```

```

sess.version = SNMP_VERSION_2c;
sess.peername = strdup(hp->name);
sess.community = strdup(hp->community);
sess.community_len = strlen(sess.community);
sess.callback = async_response; /* default callback */
sess.callback_magic = hs;
if (!(hs->sess = snmp_open(&sess))) {
    snmp_perror("snmp_open");
    continue;
}
hs->current_oid = oids;
req = snmp_pdu_create(SNMP_MSG_GET); /* send the first GET */
snmp_add_null_var(req, hs->current_oid->Oid, hs->current_oid->OidLen);
if (snmp_send(hs->sess, req))
    active_hosts++;
else {
    snmp_perror("snmp_send");
    snmp_free_pdu(req);
}
}

while (active_hosts) { /* loop while any active hosts */
    int fds = 0, block = 1;
    fd_set fdset;
    struct timeval timeout;

    FD_ZERO(&fdset);
    snmp_select_info(&fds, &fdset, &timeout, &block);
    fds = select(fds, &fdset, NULL, NULL, block ? NULL : &timeout);
    if (fds < 0) {
        perror("select failed");
        exit(1);
    }
    if (fds)
        snmp_read(&fdset);
    else
        snmp_timeout();
}

for (hp = hosts, hs = sessions; hp->name; hs++, hp++) { /* cleanup */
    if (hs->sess) snmp_close(hs->sess);
}
}

/*****/
int main (int argc, char **argv)
{
    initialize();
    printf("----- asynchronous -----\\n");
    asynchronous();
    return 0;
}

```

5.6 结果输出

6 DNS 服务

虚拟机运行 linux 系统，dns 服务程序应选择 bind（named 进程）
DNS 服务配置见 PPT

7 网络拓扑结构监控(visjs)

Vis.js 包括 4 个全局对象，分别为：

- (1) vis.Network(containers,data,options); 网络拓扑图
- (2) vis.Timeline(containers,items,options); 基于时间的线性图
- (3) vis.Graph3d(containers,data,options); 三维图
- (4) vis.Graph2d(containers,dataset,options); 二维图

7.1 组成对象-Modules

7.1.1 Network-configure 对象(对象->选项->属性/C:结构体->成员项): 定义 HTML 的画布(canvas)对象

1) 示例：

```
var options = {
  configure: {
    enabled: true,
    filter: 'nodes,edges',
    container: undefined,
    showButton: true
  }
}
network.setOptions(options);
```

2) 选项说明

Name	Type	Default	Description
enabled	Boolean	true	Toggle the configuration interface on or off. This is an optional parameter. If left undefined and any of the other properties of this object are defined, this will be set to true.
filter	String, Array, Boolean, Function	true	When a boolean, true gives you all options, false will not show any. If a string is supplied, any combination of the following is allowed: nodes, edges, layout, interaction, manipulation, physics, selection, renderer. Feel free to come up with a fun separating character. Finally, when supplied an array of strings, any of the previously mentioned fields are accepted. When supplying a function, you receive two arguments. The option and the path of the option within the options object. If it returns true, the options will be shown in the configurator. Example: <pre>function (option, path) { return path.indexOf('physics') !== -1; }</pre> This only shows the physics options.
container	DOM element	undefined	This allows you to put the configure list in another HTML container than below the network.
showButton	Boolean	true	Show the generate options button at the bottom of the configurator.

7.1.2 Network-edges 对象(对象->选项->属性/C:结构体->成员项): 边的生成和删除，它含有全局边选项，它是一个对象

1) 示例

```
var options = {
  edges: {
    arrows: 'to, from',
    color: 'red',
    font: '12px arial #ff0000',
    scaling: {
      label: true,
    },
    shadow: true,
    smooth: true,
  }
}
```

edges 全局对象，用来给所有 edge 赋值
属性,结构体的成员项

```
network.setOptions(options);
```

2) 说明: edges 对象是全局对象，edge 是单个对象，每一个单个对象有一个'id'来区分。

edges 是一个对象，定义的选项都被包含于该对象中。所有的选项可以按每 edge 设置，那么每一个 edge 就需要使用 'id' 来做区分。如果选项被定义在全局 edges 对象中，那么将应用于所有的 edge 对象中。如果一个 edge 有其自己的选项，那么，这些选项将替换全局选项

当一个 edge 设定选项，则意味着该选项将覆盖全局选项中的该属性，如果将一个 edge 的选项设置为 null，则

默认的行为是采用默认值

Name	Type	Default	Description
▸ arrows	Object or String	undefined	To draw an arrow with default settings a string can be supplied. For example: <code>arrows: 'to, from, middle'</code> or <code>'to;from'</code> , any combination with any separating symbol is fine. If you want to control the size of the arrowheads, you can supply an object.
arrowStrikethrough	Boolean	true	When false, the edge stops at the arrow. This can be useful if you have thick lines and you want the arrow to end in a point. Middle arrows are not affected by this.
▸ chosen	Object or Boolean	true	When true, selecting or hovering on an edge will change it and its label's characteristics according to the default. When false, no change to the edge or its label will occur when the edge is chosen. If an object is supplied, finer-grained adjustment of edge and label characteristics is available when an edge is chosen.
▸ color	Object or String	Object	The color object contains the color information of the edge in every situation. When the edge only needs a single color, a color value like <code>'rgb(120,32,14)'</code> , <code>'#ffffff'</code> or <code>'red'</code> can be supplied instead of an object.
dashes	Array or Boolean	false	When true, the edge will be drawn as a dashed line. You can customize the dashes by supplying an Array. Array format: Array of numbers, gap length, dash length, gap length, dash length, ... etc. The array is repeated until the distance is filled. <i>When using dashed lines in IE versions older than 11, the line will be drawn straight, not smooth.</i>
▸ font	Object or String	false	This object defines the details of the label. A shorthand is also supported in the form <code>'size face color'</code> for example: <code>'14px arial red'</code> .
from	Number or String		Edges are between two nodes, one to and one from. This is where you define the from node. You have to supply the corresponding node ID. This naturally only applies to individual edges.
hidden	Boolean	false	When true, the edge is not drawn. It is part still part of the physics simulation however!
hoverWidth	Number or Function	0.5	Assuming the hover behaviour is enabled in the interaction module , the hoverWidth determines the width of the edge when the user hovers over it with the mouse. If a number is supplied, this number will be added to the width. Because the width can be altered by the value and the scaling functions, a constant multiplier or added value may not give the best results. To solve this, you can supply a function. Example: <pre>var options: { edges: { hoverWidth: function (width) {return width+1;} } }</pre> It receives the <code>Number width</code> of the edge. In this simple example we add 1 to the width. You can tailor the logic in the function as long as it returns a <code>Number</code> .
id	String	undefined	The id of the edge. The id is optional for edges. When not supplied, an UUID will be assigned to the edge. This naturally only applies to individual edges.
label	String	undefined	The label of the edge. HTML does not work in here because the network uses HTML5 Canvas.

labelHighlightBold	Boolean	true	Determines whether or not the label becomes bold when the edge is selected.
length	Number	undefined	The physics simulation gives edges a spring length. This value can override the length of the spring in rest.
physics	Boolean	true	When true, the edge is part of the physics simulation. When false, it will not act as a spring.
▸ scaling	Object	Object	If the value option is specified, the width of the edges will be scaled according to the properties in this object. Keep in mind that when using scaling, the width option is neglected.
selectionWidth	Number or Function	1	The selectionWidth determines the width of the edge when the edge is selected. If a number is supplied, this number will be added to the width. Because the width can be altered by the value and the scaling functions, a constant multiplier or added value may not give the best results. To solve this, you can supply a function. Example: <pre>var options: { edges: { selectionWidth: function (width) {return width*2;} } }</pre> It receives the Number width of the edge. In this simple example multiply the width by 2. You can tailor the logic in the function as long as it returns a Number.
selfReferenceSize	Number	false	When the to and from nodes are the same, a circle is drawn. This is the radius of that circle.
▸ shadow	Object or Boolean	Object	When true, the edge casts a shadow using the default settings. This can be further refined by supplying an object.
▸ smooth	Object or Boolean	Object	When true, the edge is drawn as a dynamic quadratic bezier curve. The drawing of these curves takes longer than that of straight curves but it looks better. There is a difference between dynamic smooth curves and static smooth curves. The dynamic smooth curves have an invisible support node that takes part in the physics simulation. If you have a lot of edges, you may want to consider picking a different type of smooth curves then dynamic for better performance.
title	String	undefined	The title is shown in a pop-up when the mouse moves over the edge.
to	Number or String		Edges are between two nodes, one to and one from. This is where you define the to node. You have to supply the corresponding node ID. This naturally only applies to individual edges.
value	Number	undefined	When a value is set, the edges' width will be scaled using the options in the scaling object defined above.
width	Number	1	The width of the edge. If value is set, this is not used.
▸ widthConstraint	Number, Boolean or Object	false	If false, no widthConstraint is applied. If a number is specified, the maximum width of the edge's label is set to the value. The edge's label's lines will be broken on spaces to stay below the maximum.

7.1.3 Network-groups 对象(对象->选项->属性/C:结构体->成员项): 包含组对象及组对象的选项(属性), 画布(canvas)中准备包含的对象均放在一个 groups 对象

如: nodes.push({id: 1000, x: x, y: y, label: 'Internet', group: 'internet', value: 1, fixed: true, physics:false});
nodes.push({id: 1001, x: x, y: y + step, label: 'Switch', group: 'switch', value: 1, fixed: true, physics:false});
nodes.push({id: 1002, x: x, y: y + 2 * step, label: 'Server', group: 'server', value: 1, fixed: true, physics:false});
nodes.push({id: 1003, x: x, y: y + 3 * step, label: 'Computer', group: 'desktop', value: 1, fixed: true, physics:false});
nodes.push({id: 1004, x: x, y: y + 4 * step, label: 'Smartphone', group: 'mobile', value: 1, fixed: true, physics:false});
groups: {
 'switch': {
 shape: 'triangle',
 color: '#FF9900' // orange
 },
 'desktop': {
 shape: 'dot',
 color: "#2B7CE9" // blue
 },
 'mobile': {
 shape: 'dot',
 color: "#5A1E5C" // purple
 }
}

```

    },
    server: {
      shape: 'square',
      color: "#C5000B" // red
    },
    internet: {
      shape: 'square',
      color: "#109618" // green
    }
  }
}

```

1) 示例

```

var options = {
  groups: {
    useDefaultGroups: true,
    myGroupId: { /*node options*/
    }
  }
}
network.setOptions(options);

```

2) 说明

Name	Type	Default	Description
useDefaultGroups	Boolean	true	If your nodes have groups defined that are not in the Groups module, the module loops over the groups it does have, allocating one for each unknown group. When all are used, it goes back to the first group. By setting this to false, the default groups will not be used in this cycle.
group*	Object		You can add multiple groups containing styling information that applies to a certain subset of groups. All options described in the nodes module that make sense can be used here (you're not going to set the same id or x,y position for a group of nodes). Example: <pre> var nodes = [{id:1, group:'myGroup', label:"I'm in a custom group called 'myGroup'!"}] var options = { groups: { myGroup: {color:{background:'red'}, borderWidth:3} } } </pre> *) the option is not called group, as shown by the example but can be any custom id, except for 'useDefaultGroups'.

7.1.4 Network-interaction 对象(对象->选项->属性/C:结构体->成员项): 用户与 Network 图中节点交互, 即将处理鼠标和触模板做与 buttons、popups 一样的事件生成器

1) 示例

```

var options = {
  interaction: {
    keyboard: false
  }
}

network.setOptions(options);

```

2) 说明

Name	Type	Default	Description
dragNodes	Boolean	true	When true, the nodes that are not fixed can be dragged by the user.
dragView	Boolean	true	When true, the view can be dragged around by the user.
hideEdgesOnDrag	Boolean	false	When true, the edges are not drawn when dragging the view. This can greatly speed up responsiveness on dragging, improving user experience.
hideNodesOnDrag	Boolean	false	When true, the nodes are not drawn when dragging the view. This can greatly speed up responsiveness on dragging, improving user experience.
hover	Boolean	false	When true, the nodes use their hover colors when the mouse moves over them.
hoverConnectedEdges	Boolean	true	When true, on hovering over a node, it's connecting edges are highlighted.
▸ keyboard	Object or Boolean	Object	When true, the keyboard shortcuts are enabled with the default settings. For further customization, you can supply an object.
multiselect	Boolean	false	When true, a longheld click (or touch) as well as a control-click will add to the selection.
navigationButtons	Boolean	false	When true, navigation buttons are drawn on the network canvas. These are HTML buttons and can be completely customized using CSS.
selectable	Boolean	true	When true, the nodes and edges can be selected by the user.
selectConnectedEdges	Boolean	true	When true, on selecting a node, its connecting edges are highlighted.
tooltipDelay	Number	300	When nodes or edges have a defined 'title' field, this can be shown as a pop-up tooltip. The tooltip itself is an HTML element that can be fully styled using CSS. The delay is the amount of time in milliseconds it takes before the tooltip is shown.
zoomView	Boolean	true	When true, the user can zoom in.

7.1.5 Network-layout 对象((对象->选项->属性/C:结构体->成员项)): 管理初始化和层叠样式: 实现激活、大小缩放、聚焦

1) 选项

```
var options = {
  layout: {
    hierarchical: true
  }
}
network.setOptions(options);
```

2) 说明

Name	Type	Default	Description
randomSeed	Number	undefined	When NOT using the hierarchical layout, the nodes are randomly positioned initially. This means that the settled result is different every time. If you provide a random seed manually, the layout will be the same every time. Ideally you try with an undefined seed, reload until you are happy with the layout and use the <code>getSeed()</code> method to ascertain the seed.
improvedLayout	Boolean	true	When enabled, the network will use the Kamada Kawai algorithm for initial layout. For networks larger than 100 nodes, clustering will be performed automatically to reduce the amount of nodes. This can greatly improve the stabilization times. If the network is very interconnected (no or few leaf nodes), this may not work and it will revert back to the old method. Performance will be improved in the future.
▸ hierarchical	Object or Boolean	Object	When true, the layout engine positions the nodes in a hierarchical fashion using default settings. For customization you can supply an object.

7.1.6 Network-manipulation:API 或可选 GUI, 改变网络中的数据 (如丢一个 ICMP 报文节点由绿色变为粉红、丢 2 个 ICMP

报文节点由由粉红色变更为红色、丢 3 个 ICMP 报文节点由红色变更为暗红色)

1) 选项示例

```
var options = {
  manipulation: {
    enabled: false,
    initiallyActive: false,
    addNode: true,
    addEdge: true,
    editNode: undefined,
    editEdge: true,
    deleteNode: true,
    deleteEdge: true,
    controlNodeStyle:{          // all node options are valid.
  }
}
}
network.setOptions(options);
```

2) 选项说明

Name	Type	Default	Description
enabled	<i>Boolean</i>	<i>false</i>	Toggle the manipulation system on or off. Even when false, the manipulation API through the methods will still work. This property is optional. If you define any of the options below and enabled is undefined, this will be set to true.
initiallyActive	<i>Boolean</i>	<i>true</i>	Toggle whether the toolbar is visible initially or if only the edit button is visible initially.
addNode	<i>Boolean</i> or <i>Function</i>	<i>true</i>	You can use these options to switch certain functionalities on or off of attach a handler function to them. These functions are called before the action is performed. If a node is going to be added through the manipulation system, the addNode function will be called first. With this, you can provide a gui for your users, abort the process or anything else you want to do. For all except the editNode functionality, these handler functions are optional. When you supply a boolean, you only toggle the 'add node' button on the GUI of the manipulation system. The lack of handling function could effect the API when using the methods. When a function is supplied, it will be called when the user clicks the canvas in 'addNode' mode. This function will receive two variables: the properties of the node that can be created and a callback function. If you call the callback function with the properties of the new node, the node will be added. Example: <pre>var options = { manipulation: { addNode: function(nodeData,callback) { nodeData.label = 'hello world'; callback(nodeData); } } }</pre> This function changes the label of the new node into 'hello world'. If you do not want the node created, do not call the callback function or call the callback function <i>null</i> or no argument.

addEdge	Boolean or Function	true	If boolean, toggle the availability of the 'addEdge' button in the GUI, the API through the methods will still work except (obviously) there will be no handler function. When a function is supplied, it will be called when the user drags the new edge from one node to the next in 'addEdge' mode. This function will receive two variables: the properties of the edge that can be created and a callback function. If you call the callback function with the properties of the new edge, the edge will be added. Example: <pre>var options = { manipulation: { addEdge: function(edgeData,callback) { if (edgeData.from === edgeData.to) { var r = confirm("Do you want to connect the node to itself?"); if (r === true) { callback(edgeData); } } else { callback(edgeData); } } } }</pre> This example code will show a popup if you connect a node to itself to ask you if that was what you wanted. If you do not want the edge created, do not call the callback function or call the callback function <code>null</code> or no argument.
editNode	Function	undefined	Editing of nodes is only possible when a handling function is supplied. If this is not the case, editing of nodes will be disabled. The function will be called when a node is selected and the 'Edit Node' button on the toolbar is pressed. This function will be called like the <code>addNode</code> function with the node's data and a callback function.
editEdge	Boolean or Function	true	If boolean, toggle the editing of edges in the GUI. If a function is supplied, it will be called when an edge is selected and the 'Edit Edge' button on the toolbar is pressed. Initially, the endpoints of the edge may be dragged to connect to a different node, then the function will be called in the same way the <code>addEdge</code> function is called. If an object, if a function is given for the <code>'editWithoutDrag'</code> property, the function will be called immediately (without dragging any endpoints) in the same way the <code>addEdge</code> function is called. If the callback is not performed, the edge will remain hanging where it was released. To cancel, call the callback function with <code>null</code> as argument or without arguments.
deleteNode	Boolean or Function	true	If boolean, toggle the deletion of nodes in the GUI. If function, it will be called when a node is selected and the 'Delete selected' button is pressed. When using a function, it will receive a callback and an object with an array of selected nodeIds and an array of selected edges Ids. These are the items that will be deleted if the callback is performed.
deleteEdge	Boolean or Function	true	If boolean, toggle the deletion of edges in the GUI. If function, it will be called when an edge is selected and the 'Delete selected' button is pressed. When using a function, it will receive a callback and an object with an array of selected nodeIds (empty) and an array of selected edges Ids. These are the items that will be deleted if the callback is performed.

controlNodeStyle	Object	Object	You can supply any styling information you'd like here. All fields described in the nodes module are allowed except obviously for id, x, y and fixed. Default: <pre> { shape: 'dot', size: 6, color: { background: '#ff0000', border: '#3c3c3c', highlight: { background: '#07f968', border: '#3c3c3c' } }, borderWidth: 2, borderWidthSelected: 2 } </pre>
------------------	--------	--------	---

7.1.7 Node 对象(对象->选项->属性/C:结构体->成员项):节点的生成与删除，也包括全局节点选项与样式

全局对象称为 `nodes`，`node` 也可以单独定义，为了区分，每一个节点有一个'id'(全局 `nodes` 没有 id 编号)

选项可以给应用于全局节点 `nodes`，也可以被应用于每节点，如果一个节点被指定一个选项，则被认为采用该值覆盖全局节点中的一个属性对应的值，若该属性为 `null`，则默认使用默认值

1) 示例

```

var options = {
  nodes: {
    color: '#ff0000',
    fixed: false,
    font: '12px arial red',
    scaling: {
      label: true
    },
    shadow: true
  }
}
node.color=
network.setOptions(options);

```

```

struct node{
}variable
variable.color

```

2) 说明

Name	Type	Default	Description
borderWidth	Number	1	The width of the border of the node.
borderWidthSelected	Number	2	The width of the border of the node when it is selected. When undefined, the borderWidth * 2 is used.
brokenImage	String	undefined	When the shape is set to <code>image</code> or <code>circularImage</code> , this option can be an URL to a backup image in case the URL supplied in the image option cannot be resolved.
▸ chosen	Object or Boolean	true	When true, selecting or hovering on a node will change it and its label's characteristics according the default. When false, no change to the node or its label will occur when the node is chosen. If an object is supplied, finer-grained adjustment of node and label characteristics is available when a node is chosen.
▸ color	Object or String	Object	The color object contains the color information of the node in every situation. When the node only needs a single color, a color value like <code>'rgba(120,32,14,1)'</code> , <code>'#ffffff'</code> or <code>'red'</code> can be supplied instead of an object.
▸ fixed	Object or Boolean	Object	When true, the node will not move but IS part of the physics simulation. When defined as an object, movement in either X or Y direction can be disabled.
▸ font	Object or String	false	This object defines the details of the label. A shorthand is also supported in the form <code>'size face color'</code> for example: <code>'14px arial red'</code> .
group	String	undefined	When not <code>undefined</code> , the node will belong to the defined group. Styling information of that group will apply to this node. Node specific styling overrides group styling.
▸ heightConstraint	Number, Boolean or Object	false	If false, no heightConstraint is applied. If a number is specified, the value is used as the minimum height of the node. The node's height will be set to the minimum if less than the value.
hidden	Boolean	false	When true, the node will not be shown. It will still be part of the physics simulation though!
▸ icon	Object	Object	These options are only used when the shape is set to <code>icon</code> .
id	String	undefined	The id of the node. The id is mandatory for nodes and they have to be unique. This should obviously be set per node, not globally.
▸ image	Object or String	undefined	When the shape is set to <code>image</code> or <code>circularImage</code> , this option should be the URL to an image. If the image cannot be found, the brokenImage option can be used.
label	String	undefined	The label is the piece of text shown in or under the node, depending on the shape.
labelHighlightBold	Boolean	true	Determines whether or not the label becomes bold when the node is selected.
level	Number	undefined	When using the hierarchical layout, the level determines where the node is going to be positioned.
▸ margin	Object or Number	5	If a number is specified, the margins of the label are set to that value on all sides. These options are only used when the shape is set to <code>box</code> , <code>circle</code> , <code>database</code> , <code>icon</code> or <code>text</code> .

mass	Number	1	The BarnesHut physics model (which is enabled by default) is based on an inverted gravity model. By increasing the mass of a node, you increase its repulsion. Values between 0 and 1 are not recommended. Negative or zero values are not allowed. These will generate a console error and will be set to 1.
physics	Boolean	true	When false, the node is not part of the physics simulation. It will not move except for from manual dragging.
▸ scaling	Object	Object	If the <code>value</code> option is specified, the size of the nodes will be scaled according to the properties in this object. All node shapes can be scaled, but some only when label scaling is enabled as their size is based on the size of the label. Only scalable when label scaling is enabled are: <code>ellipse</code>, <code>circle</code>, <code>database</code>, <code>box</code>, <code>text</code>. Always scalable are: <code>image</code>, <code>circularImage</code>, <code>diamond</code>, <code>dot</code>, <code>star</code>, <code>triangle</code>, <code>triangleDown</code>, <code>hexagon</code>, <code>square</code> and <code>icon</code>. Keep in mind that when using scaling, the <code>size</code> option is neglected.
▸ shadow	Object or Boolean	Object	When true, the node casts a shadow using the default settings. This can be further refined by supplying an object.
shape	String	'ellipse'	The shape defines what the node looks like. There are two types of nodes. One type has the label inside of it and the other type has the label underneath it. The types with the label inside of it are: <code>ellipse</code>, <code>circle</code>, <code>database</code>, <code>box</code>, <code>text</code>. The ones with the label outside of it are: <code>image</code>, <code>circularImage</code>, <code>diamond</code>, <code>dot</code>, <code>star</code>, <code>triangle</code>, <code>triangleDown</code>, <code>hexagon</code>, <code>square</code> and <code>icon</code>.
▸ shapeProperties	Object	Object	This object contains configuration for specific shapes
size	Number	25	The size is used to determine the size of node shapes that do not have the label inside of them. These shapes are: <code>image</code>, <code>circularImage</code>, <code>diamond</code>, <code>dot</code>, <code>star</code>, <code>triangle</code>, <code>triangleDown</code>, <code>hexagon</code>, <code>square</code> and <code>icon</code>.
title	String or Element	undefined	Title to be displayed when the user hovers over the node. The title can be an HTML element or a string containing plain text or HTML.
value	Number	undefined	When a value is set, the nodes will be scaled using the options in the scaling object defined above.
▸ widthConstraint	Number, Boolean or Object	false	If false, no widthConstraint is applied. If a number is specified, the minimum and maximum widths of the node are set to the value. The node's label's lines will be broken on spaces to stay below the maximum and the node's width will be set to the minimum if less than the value.
x	Number	undefined	This gives a node an initial x position. When using the hierarchical layout, either the x or y position is set by the layout engine depending on the type of view. The other value remains untouched. When using stabilization, the stabilized position may be different from the initial one. To lock the node to that position use the physics or fixed options.
y	Number	undefined	This gives a node an initial y position. When using the hierarchical layout, either the x or y position is set by the layout engine depending on the type of view. The other value remains untouched. When using stabilization, the stabilized position may be different from the initial one. To lock the node to that position use the physics or fixed options.

7.1.8 physics 对象(对象->选项->属性/C:结构体->成员项): 模拟实现移动 node 和 edge 的模块, 动画效果同时也管理 node 与 edge 的稳定性

1) 示例

```
var options = {
  physics: {
```

```

        stabilization: false
    }
}
network.setOptions(options);

```

2) 选项说明

Name	Type	Default	Description
enabled	Boolean	true	Toggle the physics system on or off. This property is optional. If you define any of the options below and enabled is undefined, this will be set to true.
▸ barnesHut	Object	Object	BarnesHut is a quadtree based gravity model. This is the fastest, default and recommended solver for non-hierarchical layouts.
▸ forceAtlas2Based	Object	Object	Force Atlas 2 has been developed by Jacomi et al (2014) for use with Gephi. The forceAtlas2Based solver makes use of some of the equations provided by them and makes use of the barnesHut implementation in vis. The main differences are the central gravity model, which is here distance independent, and the repulsion being linear instead of quadratic. Finally, all node masses have a multiplier based on the amount of connected edges plus one.
▸ repulsion	Object	Object	The repulsion model assumes nodes have a simplified repulsion field around them. It's force linearly decreases from 1 (at 0.5*nodeDistance and smaller) to 0 (at 2*nodeDistance).
▸ hierarchicalRepulsion	Object	Object	This model is based on the repulsion solver but the levels are taken into account and the forces are normalized.
maxVelocity	Number	50	The physics module limits the maximum velocity of the nodes to increase the time to stabilization. This is the maximum value.
minVelocity	Number	0.1	Once the minimum velocity is reached for all nodes, we assume the network has been stabilized and the simulation stops.
solver	String	'barnesHut'	You can select your own solver. Possible options: 'barnesHut', 'repulsion', 'hierarchicalRepulsion', 'forceAtlas2Based'. When setting the hierarchical layout, the hierarchical repulsion solver is automatically selected, regardless of what you fill in here.
▸ stabilization	Object Boolean	Object	When true, the network is stabilized on load using default settings. If false, stabilization is disabled. To further customize this, you can supply an object.
timestep	Number	0.5	The physics simulation is discrete. This means we take a step in time, calculate the forces, move the nodes and take another step. If you increase this number the steps will be too large and the network can get unstable. If you see a lot of jittery movement in the network, you may want to reduce this value a little.
adaptiveTimestep	Boolean	true	If this is enabled, the timestep will intelligently be adapted (only during the stabilization stage if stabilization is enabled!) to greatly decrease stabilization times. The timestep configured above is taken as the minimum timestep. This can be further improved by using the improvedLayout algorithm.

7.2 全局选项-options 对象 ((对象->选项->属性/C:结构体->成员项))

```

var options = {
  autoResize: true,
  height: '100%',
  width: '100%',
  locale: 'en',           语言（小写）_国家代码（大写）
  locales: locales,
  clickToUse: false,
  configure: {...},      // defined in the configure module.
  edges: {...},          // defined in the edges module.
  nodes: {...},          // defined in the nodes module.
  groups: {...},         // defined in the groups module.
  layout: {...},         // defined in the layout module.

```

```

interaction: {...}, // defined in the interaction module.
manipulation: {...}, // defined in the manipulation module.
physics: {...}, // defined in the physics module.
}
network.setOptions(options);

```

Name	Type	Default	Description
autoResize	Boolean	true	If true, the Network will automatically detect when its container is resized, and redraw itself accordingly. If false, the Network can be forced to repaint after its container has been resized using the function <code>redraw()</code> and <code>setSize()</code> .
width	String	'100%'	the width of the canvas. Can be in percentages or pixels (ie. '400px').
height	String	'100%'	the height of the canvas. Can be in percentages or pixels (ie. '400px').
locale	String	'en'	Select the locale. By default, the language is English.
locales	Object	defaultLocales	Locales object. By default 'en', 'de', 'es', 'it', 'nl', 'pt-br', 'ru' are supported. Take a look at the locales section below for more explanation on how to customize this.
clickToUse	Boolean	false	When a Network is configured to be <code>clickToUse</code> , it will react to mouse and touch events only when active. When active, a blue shadow border is displayed around the Network. The network is set active by clicking on it, and is changed to inactive again by clicking outside the Network or by pressing the ESC key.
configure	Object	Object	All options in this object are explained in the configure module .
edges	Object	Object	All options in this object are explained in the edges module .
nodes	Object	Object	All options in this object are explained in the nodes module .
groups	Object	Object	All options in this object are explained in the groups module .
layout	Object	Object	All options in this object are explained in the layout module .
interaction	Object	Object	All options in this object are explained in the interaction module .
manipulation	Object	Object	All options in this object are explained in the manipulation module .
physics	Object	Object	All options in this object are explained in the physics module .

7.3、public API

7.3.1 Global methods for network

1) `destroy()`: Remove the network from the DOM and remove all Hammer bindings and references

无返回值

2) `setData({nodes:vis dataset/array, edges:vis dataset/Array})`

Override all the data in the network. If Stabilization is enabled in the physics module, the network will stabilize again

This method is also performed when first initializing the network

3) `setOptions(Object options)`: set the options

4) `on(String event name, function callback)`:注册

5) `off(string event name, function callback)` 解除注册

6) `once(String event name, Function callback)` 其注册的事件与 `callback` 对应关系只保持一次

7.3.2 与画布(canvas)有关的方法

1) `canvasToDOM({x:Number, y:Number})`

This function converts canvas coordinates to coordinates on the DOM, input and output are in the form of `{x:Number, y:Number}`. The DOM values are relative the network container

2) `DOMtoCanvas({x:Number, y:Number})`

3) `redraw()` Redraw the network

4) setSize(String width, String height): set the size of the canvas, This is automatically done on a windows resize

7.3.3 Clustering

1) cluster(Object options) 生成一个 cluster

2) clusterByConnection(string nodeId, [Object options])

This method looks at the provided node and makes a cluster of it and all it's connected nodes. The behaviour can be customized by providing the options object. All options of this object are explained below. The joinCondition is only presented with the connected nodes.

3) clusterByHubsize([Number hubsize],[Object options])

This method checks all nodes in the network and those with a equal or higher amount of edges than specified with the **hubsize** qualify. If a hubsize is not defined, the hubsize will be determined as the average value plus two standard deviations.

For all qualifying nodes, clusterByConnection is performed on each of them. The options object is described for **clusterByConnection** and does the same here.

4) clusterByHubsize([Number hubsize],[Object options])

This method checks all nodes in the network and those with a equal or higher amount of edges than specified with the **hubsize** qualify. If a hubsize is not defined, the hubsize will be determined as the average value plus two standard deviations.

For all qualifying nodes, clusterByConnection is performed on each of them. The options object is described for **clusterByConnection** and does the same here.

5) clusterOutliers([Object options])

This method will cluster all nodes with 1 edge with their respective connected node. The options object is explained in full **below**.

6) findNode(string/Number nodeId) 返回值: array

Nodes can be in clusters. Clusters can also be in clusters. This function returns an array of nodeIds showing where the node is.

If any nodeId in the chain, especially the first passed in as a parameter, is not present in the current nodes list, an empty array is returned.

Example: cluster 'A' contains cluster 'B', cluster 'B' contains cluster 'C', cluster 'C' contains node 'fred'. **network.clustering.findNode('fred')** will return ['A','B','C','fred'].

7) getClusteredEdges(string baseEdgeId) 返回值: array

Similar to **findNode** in that it returns all the edge ids that were created from the provided edge during clustering

8) getBaseEdge(string clusteredEdgeId) 返回值: value

When a clusteredEdgeId is available, this method will return the original baseEdgeId provided in **data.edges**

ie. After clustering the 'SelectEdge' event is fired but provides only the clustered edge. This method can then be used to return the baseEdgeId.

This method is deprecated. Please use **getBaseEdges()** instead.

9) getBaseEdges(string clusteredEdgeId) 返回值: array

For the given **clusteredEdgeId**, this method will return all the original base edge id's provided in **data.edges**. For a non-

clustered (i.e. 'base') edge, `clusteredEdgeId` is returned.

Only the base edge id's are returned. All clustered edges id's under `clusteredEdgeId` are skipped, but scanned recursively to return their base id's.

10) `updateEdge(String startEdgeId, Object options)` 返回值: none

Visible edges between clustered nodes are not the same edge as the ones provided in `data.edges` passed on `network` creation. With each layer of clustering, copies of the edges between clusters are created and the previous edges are hidden, until the cluster is opened.

This method takes an `edgeId` (ie. a base `edgeId` from `data.edges`) and applies the options to it and any edges that were created from it while clustering.

Example: `network.clustering.updateEdge(originalEdge.id, {color: '#aa0000'});`

This would turn the base edge and any subsequent edges red, so when opening clusters the edges will all be the same color.

11) `updateClusteredNode(String clusteredNodeId, Object options)` 返回值: none

Clustered Nodes when created are not contained in the original `data.nodes` passed on `network` creation

This method updates the cluster node.

Example: `network.clustering.updateClusteredNode(clusteredNodeId, {shape: 'star'});`

12) `isCluster(string nodeId)` Returns: Boolean

Returns true if the node whose ID has been supplied is a cluster

13) `getNodesInCluster(String clusterNodeId)` Returns: array

Returns an array of all `nodeIds` of the nodes that would be released if you open the cluster

14) `openCluster(string nodeId, Object options)` 返回值: none

Opens the cluster, releases the contained nodes and edges, removing the cluster node and cluster edges. The options object is optional and currently supports one option, `releaseFunction`, which is a function that can be used to manually position the nodes after the cluster is opened.

```
function releaseFunction (clusterPosition, containedNodesPositions) {  
    var newPositions = {};  
    // clusterPosition = {x:clusterX, y:clusterY};  
    // containedNodesPositions = {nodeId:{x:nodeX,y:nodeY}, nodeId2,...}  
    newPositions[nodeId] = {x:newPosX, y:newPosY};  
    return newPositions;  
}
```

The `containedNodesPositions` contain the positions of the nodes in the cluster at the moment they were clustered. This function is expected to return the `newPositions`, which can be the `containedNodesPositions` (altered) or a new object. This has to be an object with keys equal to the `nodeIds` that exist in the `containedNodesPositions` and an `{x:x,y:y}` position object. For all `nodeIds` not listed in this returned object, we will position them at the location of the cluster. This is also the default behaviour when no `releaseFunction` is defined.

7.3.4 Layout

1) `getSeed()` 返回值: Number

If you like the layout of your network and would like it to start in the same way next time, ask for the seed using this method and put it in the `layout.randomSeed` option.

7.3.5 Manipulation methods to use the manipulation system without GUI

1) `enableEditMode()` 返回值 `none`

Programmatically enable the edit mode. Similar effect to pressing the edit button.

2) `disableEditMode()` 返回值 `none`

Programmatically disable the edit mode. Similar effect to pressing the close icon (small cross in the corner of the toolbar).

3) `addNodeMode()` 返回值 `none`

Go into `addNode` mode. Having edit mode or manipulation enabled is not required. To get out of this mode, call `disableEditMode()`. The callback functions defined in `handlerFunctions` still apply. To use these methods without having the manipulation GUI, make sure you set `enabled` to `false`.

4) `editNode()` returns: `none`

Edit the selected node. The explanation from `addNodeMode` applies here as well.

5) `addEdgeMode()` returns: `none`

Go into `addEdge` mode. The explanation from `addNodeMode` applies here as well.

6) `editEdgeMode()` returns: `none`

Go into `editEdge` mode. The explanation from `addNodeMode` applies here as well.

7) `deleteSelected()` returns: `none`

Delete selected. Having edit mode or manipulation enabled is not required.

7.3.6 Methods to get information on nodes and edges

1) `getPositions(array of nodeIds)` returns: `Object`

Returns the x y positions in canvas space of the nodes with the supplied `nodeIds` as an object:

```
{
  nodeId1: {x: xValue, y:yValue},
  nodeId2: {x: xValue, y:yValue},
  ...
}
```

Alternative inputs are a `String` containing a `nodeId` or nothing. When a `String` is supplied, the position of the node corresponding to the ID is returned. When nothing is supplied, the positions of all nodes are returned.

2) `storePositions()` returns: `none`

When using the `vis.DataSet` to load your nodes into the network, this method will put the X and Y positions of all nodes into that dataset. If you're loading your nodes from a database and have this dynamically coupled with the `DataSet`, you can use this to stabilize your network once, then save the positions in that database through the `DataSet` so the next time you load the nodes, stabilization will be near instantaneous.

If the nodes are still moving and you're using dynamic smooth edges (which is on by default), you can use the option `stabilization.onlyDynamicEdges` in the `physics module` to improve initialization time.

This method does not support clustering. At the moment it is not possible to cache positions when using clusters since they cannot be correctly initialized from just the positions.

3) `moveNode(nodeId, Number x, Number y)` returns: `none`

You can use this to programmatically move a node. The supplied x and y positions have to be in canvas space!

4) `getBoundingBox(String nodeId)` `returns:Object`

Returns a bounding box for the node including label in the format:

```
{
  top: Number,
  left: Number,
  right: Number,
  bottom: Number
}
```

These values are in canvas space.

5) `getConnectedNodes(String nodeId or edgeId, [string direction])` `returns:Array`

Returns an array of nodeIds of all the nodes that are directly connected to this node or edge.

For a node id, returns an array with the id's of the connected nodes.

If optional parameter **direction** is set to string *'from'*, only parent nodes are returned.

If **direction** is set to *'to'*, only child nodes are returned.

Any other value or **undefined** returns both parent and child nodes.

For an edge id, returns an array: **[fromId, toId]**. Parameter *direction* is ignored for edges.

6) `getConnectedEdges(string nodeId)` `returns:array`

Returns an array of edgeIds of the edges connected to this node.

7.3.7 Physics methods to control when the simulation should run

1) `startSimulation()` `returns:none`

Start the physics simulation. This is normally done whenever needed and is only really useful if you stop the simulation yourself and wish to continue it afterwards.

2) `stopSimulation()` `returns:none`

This stops the physics simulation and triggers a **stabilized** event. It can be restarted by dragging a node, altering the dataset or calling **startSimulation()**.

3) `stabilize([iterations])` `returns:none`

You can manually call stabilize at any time. All the stabilization options above are used. You can optionally supply the number of iterations it should do.

7.3.8 Selection methods for nodes and edges

1) `getSelection()` `returns:Object`

Returns an object with selected nodes and edges ids like this:

```
{
  nodes: [Array of selected nodeIds],
  edges: [Array of selected edgeIds]
}
```

2) `getSelectedNodes()` `returns:array`

Returns an array of selected node ids like so: **[nodeId1, nodeId2, ..]**.

3) `getSelectedEdges()` `returns:array`

Returns an array of selected edge ids like so: **[edgeId1, edgeId2, ..]**.

4) getNodeAt(x: xPos DOM, y: yPos DOM) returns: string

Returns a nodeId or undefined. The DOM positions are expected to be in pixels from the top left corner of the canvas

5) getEdgeAt(x: xPos DOM, y: yPos DOM) returns :string

Returns an edgeId or undefined. The DOM positions are expected to be in pixels from the top left corner of the canvas..

6) selectEdges(array with edgeIds) returns: none

Selects the edges corresponding to the id's in the input array. This method unselects all other objects before selecting its own objects. *Does not fire events.*

7) setSelection(Object selection, [Object options]) returns: none

Sets the selection, which must be an object like this:

```
{
  nodes: [Array of nodeIds],
  edges: [Array of edgeIds]
}
```

You can also pass only **nodes** or **edges** in **selection** object. Available options are:

```
{
  unselectAll: Boolean,
  highlightEdges: Boolean
}
```

8) unselectAll() returns: none

Unselect all objects. *Does not fire events.*

7.3.9 Methods to control the viewport for zoom and animation

1) getScale() returns: Number

Returns the current scale of the network. 1.0 is comparable to 100%, 0 is zoomed out infinitely.

2) getViewPosition() returns: Object

Returns the current central focus point of the view in the form: **{ x: {Number}, y: {Number} }**

3) fit([Object options]) returns: none

Zooms out so all nodes fit on the canvas. You can supply options to customize this:

```
{
  nodes: [Array of nodeIds],
  animation: { // -----> can be a boolean too!
    duration: Number
    easingFunction: String
  }
}
```

The nodes can be used to zoom to fit only specific nodes in the view.

The other options are explained in the **moveTo()** description below. All options are optional for the fit method.

4) focus(String nodeId, [Object options]) returns :none

You can focus on a node with this function. What that means is the view will lock onto that node, if it is moving, the view will also move accordingly. If the view is dragged by the user, the focus is broken. You can supply options to customize the effect:

```

{
  scale: Number,
  offset: {x:Number, y:Number}
  locked: boolean
  animation: { // -----> can be a boolean too!
    duration: Number
    easingFunction: String
  }
}

```

All options except for locked are explained in the `moveTo()` description below. Locked denotes whether or not the view remains locked to the node once the zoom-in animation is finished. Default value is true. The options object is optional in the focus method.

5) moveTo(Object options) returns:none

You can animate or move the camera using the moveTo method. Options are:

```

{
  position: {x:Number, y:Number},
  scale: Number,
  offset: {x:Number, y:Number}
  animation: { // -----> can be a boolean too!
    duration: Number
    easingFunction: String
  }
}

```

The position (in canvas units!) is the position of the central focus point of the camera. The scale is the target zoomlevel. Default value is 1.0. The offset (in DOM units) is how many pixels from the center the view is focussed. Default value is {x:0,y:0}. For animation you can either use a Boolean to use it with the default options or disable it or you can define the duration (in milliseconds) and easing function manually. Available are: `linear`, `easeInQuad`, `easeOutQuad`, `easeInOutQuad`, `easeInCubic`, `easeOutCubic`, `easeInOutCubic`, `easeInQuart`, `easeOutQuart`, `easeInOutQuart`, `easeInQuint`, `easeOutQuint`, `easeInOutQuint`. You will have to define at least a scale, position or offset. Otherwise, there is nothing to move to.

6) releaseNode() returns:none

Programmatically release the focused node

7.3.10 Methods to use with the configurator module

1) getOptionsFromConfigurator() returns:Object

If you use the configurator, you can call this method to get an options object that contains all differences from the default options caused by users interacting with the configurator.

7.4 事件

Public API 中涉及的事件。这些事件被交互的对象所触发，涉及至用户的输入

Name	Properties	Description
<i>Events triggered by human interaction, selection, dragging etc.</i>		
click	Object	Fired when the user clicks the mouse or taps on a touchscreen device. Passes an object with properties structured as: <pre>{ nodes: [Array of selected nodeIds], edges: [Array of selected edgeIds], event: [Object] original click event, pointer: { DOM: {x:pointer_x, y:pointer_y}, canvas: {x:canvas_x, y:canvas_y} } }</pre> This is the structure common to all events. Specifically for the click event, the following property is added: <pre>{ ... items: [Array of click items], }</pre> Where the click items can be: <pre>{nodeId:NodeId} // node with given id clicked on {nodeId:NodeId labelId:0} // label of node with given id clicked on {edgeId:EdgeId} // edge with given id clicked on {edgeId:EdgeId, labelId:0} // label of edge with given id clicked on</pre> The order of the <code>items</code> array is descending in z-order. Thus, to get the topmost item, get the value at index 0.

doubleClick	same as <i>click</i> .	Fired when the user double clicks the mouse or double taps on a touchscreen device. Since a double click is in fact 2 clicks, 2 click events are fired, followed by a double click event. If you do not want to use the click events if a double click event is fired, just check the time between click events before processing them.
oncontext	same as <i>click</i> .	Fired when the user click on the canvas with the right mouse button. The right mouse button does not select by default. You can use the method <code>getNodeAt</code> to select the node if you want.
hold	same as <i>click</i> .	Fired when the user clicks and holds the mouse or taps and holds on a touchscreen device. A click event is also fired in this case.
release	same as <i>click</i> .	Fired after drawing on the canvas has been completed. Can be used to draw on top of the network.
select	same as <i>click</i> .	Fired when the selection has changed by user action. This means a node or edge has been selected, added to the selection or deselected. All select events are only triggered on click and hold.
selectNode	same as <i>click</i> .	Fired when a node has been selected by the user.
selectEdge	same as <i>click</i> .	Fired when a edge has been selected by the user.
deselectNode	<i>Object</i>	Fired when a node (or nodes) has (or have) been deselected by the user. The previous selection is the list of nodes and edges that were selected before the last user event. Passes an object with properties structured as: <pre> { nodes: [Array of selected nodeIds], edges: [Array of selected edgeIds], event: [Object] original click event, pointer: { DOM: {x:pointer_x, y:pointer_y}, canvas: {x:canvas_x, y:canvas_y} } }, previousSelection: { nodes: [Array of previously selected nodeIds], edges: [Array of previously selected edgeIds] } } </pre>

deselectEdge	<i>same as <code>deselectNode</code>.</i>	Fired when a edge (or edges) has (or have) been deselected by the user. The previous selection is the list of nodes and edges that were selected before the last user event.
dragStart	<i>same as <code>click</code>.</i>	Fired when starting a drag.
dragging	<i>same as <code>click</code>.</i>	Fired when dragging node(s) or the view.
dragEnd	<i>same as <code>click</code>.</i>	Fired when the drag has finished.
hoverNode	<i><code>{node: nodeId}</code></i>	Fired if the option <code>interaction:{hover:true}</code> is enabled and the mouse hovers over a node.
blurNode	<i><code>{node: nodeId}</code></i>	Fired if the option <code>interaction:{hover:true}</code> is enabled and the mouse moved away from a node it was hovering over before.
hoverEdge	<i><code>{edge: edgeId}</code></i>	Fired if the option <code>interaction:{hover:true}</code> is enabled and the mouse hovers over an edge.
blurEdge	<i><code>{edge: edgeId}</code></i>	Fired if the option <code>interaction:{hover:true}</code> is enabled and the mouse moved away from an edge it was hovering over before.
zoom	<i>Object</i>	Fired when the user zooms in or out. The properties tell you which direction the zoom is in. The scale is a number greater than 0, which is the same that you get with <code>network.getScale()</code>. When fired by clicking the zoom in or zoom out navigation buttons, the pointer property of the object passed will be null. Passes an object with properties structured as: <pre> { direction: ' + ' / ' - ', scale: Number, pointer: {x:pointer_x, y:pointer_y} } </pre>
showPopup	<i>id of item corresponding to popup</i>	Fired when the popup (tooltip) is shown.
hidePopup	<i>none</i>	Fired when the popup (tooltip) is hidden.

Events triggered the physics simulation. Can be used to trigger GUI updates.

startStabilizing	<i>none</i>	Fired when stabilization starts. This is also the case when you drag a node and the physics simulation restarts to stabilize again. Stabilization does not necessarily imply 'without showing'.
stabilizationProgress	<i>Object</i>	Fired when a multiple of the <code>updateInterval</code> number of iterations is reached. This only occurs in the 'hidden' stabilization. Passes an object with properties structured as: <pre> { iterations: Number // iterations so far, total: Number // total iterations in options } </pre>
stabilizationIterationsDone	<i>none</i>	Fired when the 'hidden' stabilization finishes. This does not necessarily mean the network is stabilized; it could also mean that the amount of iterations defined in the options has been reached.
stabilized	<i>Object</i>	Fired when the network has stabilized or when the <code>stopSimulation()</code> has been called. The amount of iterations it took could be used to tweak the maximum amount of iterations needed to stabilize the network. Passes an object with properties structured as: <pre> { iterations: Number // iterations it took } </pre>

Event triggered by the canvas.

resize	<i>Object</i>	Fired when the size of the canvas has been resized, either by a redraw call when the container div has changed in size, a <code>setSize()</code> call with new values or a <code>setOptions()</code> with new width and/or height values. Passes an object with properties structured as: <pre> { width: Number // the new width of the canvas height: Number // the new height of the canvas oldWidth: Number // the old width of the canvas oldHeight: Number // the old height of the canvas } </pre>
--------	---------------	--

Events triggered by the rendering module. Can be used to draw custom elements on the canvas.

initRedraw	<i>none</i>	Fired before the redrawing begins. The simulation step has completed at this point. Can be used to move custom elements before starting drawing the new frame.
beforeDrawing	<i>canvas context</i>	Fired after the canvas has been cleared, scaled and translated to the viewing position but before all edges and nodes are drawn. Can be used to draw behind the network.
afterDrawing	<i>canvas context</i>	Fired after drawing on the canvas has been completed. Can be used to draw on top of the network.

Event triggered by the view module.

animationFinished	<i>none</i>	Fired when an animation is finished.
-------------------	-------------	--------------------------------------

Event triggered by the configuration module.

configChange	Object	Fired when a user changes any option in the configurator. The options object can be used with the setOptions method or stringified using JSON.stringify(). You do not have to manually put the options into the network: this is done automatically. You can use the event to store user options in the database.
--------------	--------	---

7.5 引入数据

Network contains conversion utilities to import data from [Gephi](#) and graphs in the [DOT language](#).

7.5.1、从 Gephi 中引入数据

Network can import data straight from an exported json file from gephi. You can get the JSON exporter here: <https://marketplace.gephi.org/plugin/json-exporter/>. An example exists showing how to get a JSON file into Vis:

示例:

```
// load the JSON file containing the Gephi network.
var gephiJSON = loadJSON("./datasources/WorldCup2014.json"); // code in importing_from_gephi.

// you can customize the result like with these options. These are explained below. These are the default options.
var parserOptions = {
  edges: {
    inheritColors: false
  },
  nodes: {
    fixed: true,
    parseColor: false
  }
}

// parse the gephi file to receive an object containing nodes and edges in vis format.
var parsed = vis.network.convertGephi(gephiJSON, parserOptions);

// provide data in the normal fashion
var data = {
  nodes: parsed.nodes,
  edged: parsed.edges
};

// create a network
var network = new vis.Network(container, data);
```

Gephi parser options

There are a few options you can use to tell Vis what to do with the data from Gephi.

Name	Type	Default	Description
nodes.fixed	Boolean	true	When false, the nodes will move according to the physics model after import. If true, the nodes do not move at all. If set to true, the node positions have to be defined to avoid infinite recursion errors in the physics.
nodes.parseColor	Boolean	false	If true, the color will be parsed by the vis parser, generating extra colors for the borders, highlights and hover. If false, the node will be the supplied color.
edges.inheritColor	Boolean	false	When true, the color supplied by gephi is ignored and the inherit color mode is used with the global setting.

7.5.2 从 DOT 语言中引入数据

DOT:文本图形描述语言，为了支持 Graphviz

Network supports data in the DOT language. To use data in the DOT language, you can use the `vis.network.convertDot` converter to transform the DOT language into a `vis.Network` compatible nodes, edges and options objects. You can extend the options object with other options if you'd like.

例:

```
// provide data in the DOT language
var DOTstring = 'dinetwork {1 -> 1 -> 2; 2 -> 3; 2 -- 4; 2 -> 1 }';
var parsedData = vis.network.convertDot(DOTstring);
var data = {
  nodes: parsedData.nodes,
  edges: parsedData.edges
}
var options = parsedData.options;
// you can extend the options like a normal JSON variable:
options.nodes = {
  color: 'red'
}

// create a network
var network = new vis.Network(container, data, options);
```

7.6 程序示例---nodeLegend.html

```
<!DOCTYPE html>

<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Network | Images</title>

<style type="text/css">
  html, body {
    font: 10pt arial;
    padding: 0;
    margin: 0;
    width: 100%;
    height: 100%;
  }

  #mynetwork {
    width: 100%;
    height: 100%;
  }
</style>

<script type="text/javascript" src="../../dist/vis.js"></script>
<link href="../../dist/vis-network.min.css" rel="stylesheet" type="text/css" />

<script type="text/javascript">
  var nodes = null;
  var edges = null;
  var network = null;

  var  LENGTH_MAIN = 350,
      LENGTH_SERVER = 150,
      LENGTH_SUB = 50,
      WIDTH_SCALE = 2,
      GREEN = 'green',
      RED = '#C5000B',
      ORANGE = 'orange',      //GRAY = '#666666',
      GRAY = 'gray',
```

```

BLACK = '#2B1B17';

// Called when the Visualization API is loaded.
function draw() {
    nodes = [];          // Create a data table with nodes. 数组每个成员项都是 node 对象
    edges = [];          // Create a data table with links. 数组：每个成员项都是 edge 对象

    nodes.push({id: 1, label: '192.168.0.1', group: 'switch', value: 10}); 赋值，赋 node 对象属性的值
    nodes.push({id: 2, label: '192.168.0.2', group: 'switch', value: 8});
    nodes.push({id: 3, label: '192.168.0.3', group: 'switch', value: 6});
    edges.push({from: 1, to: 2, length: LENGTH_MAIN, width: WIDTH_SCALE * 6, label: '0.71 mbps'});
    edges.push({from: 1, to: 3, length: LENGTH_MAIN, width: WIDTH_SCALE * 4, label: '0.55 mbps'});
    ‘<?php echo 变量? ?>’

    // group around 2
    for (var i = 100; i <= 104; i++) {
        var value = 1;
        var width = WIDTH_SCALE * 2;
        var color = GRAY;
        var label = null;

        if (i === 103) {
            value = 5;
            width = 3;
        }
        if (i === 102) {
            color = RED;
            label = 'error';
        }

        nodes.push({id: i, label: '192.168.0.' + i, group: 'desktop', value: value});
        edges.push({from: 2, to: i, length: LENGTH_SUB, color: color, fontColor: color, width: width, label: label});
    }
    nodes.push({id: 201, label: '192.168.0.201', group: 'desktop', value: 1});
    edges.push({from: 2, to: 201, length: LENGTH_SUB, color: GRAY, width: WIDTH_SCALE});

    // group around 3
    nodes.push({id: 202, label: '192.168.0.202', group: 'desktop', value: 4});
    edges.push({from: 3, to: 202, length: LENGTH_SUB, color: GRAY, width: WIDTH_SCALE * 2});
    for (var i = 230; i <= 231; i++) {
        nodes.push({id: i, label: '192.168.0.' + i, group: 'mobile', value: 2});
        edges.push({from: 3, to: i, length: LENGTH_SUB, color: GRAY, fontColor: GRAY, width: WIDTH_SCALE});
    }

    // group around 1
    nodes.push({id: 10, label: '192.168.0.10', group: 'server', value: 10});
    edges.push({from: 1, to: 10, length: LENGTH_SERVER, color: GRAY, width: WIDTH_SCALE * 6, label: '0.92 mbps'});
    nodes.push({id: 11, label: '192.168.0.11', group: 'server', value: 7});
    edges.push({from: 1, to: 11, length: LENGTH_SERVER, color: GRAY, width: WIDTH_SCALE * 3, label: '0.68 mbps'});
    nodes.push({id: 12, label: '192.168.0.12', group: 'server', value: 3});
    edges.push({from: 1, to: 12, length: LENGTH_SERVER, color: GRAY, width: WIDTH_SCALE, label: '0.3 mbps'});

    nodes.push({id: 204, label: 'Internet', group: 'internet', value: 10});
    edges.push({from: 1, to: 204, length: 200, width: WIDTH_SCALE * 3, label: '0.63 mbps'});

    // legend
    var mynetwork = document.getElementById('mynetwork');
    var x = - mynetwork.clientWidth / 2 + 50;
    var y = - mynetwork.clientHeight / 2 + 50;
    var step = 70;
    nodes.push({id: 1000, x: x, y: y, label: 'Internet', group: 'internet', value: 1, fixed: true, physics:false});
    nodes.push({id: 1001, x: x, y: y + step, label: 'Switch', group: 'switch', value: 1, fixed: true, physics:false});
    nodes.push({id: 1002, x: x, y: y + 2 * step, label: 'Server', group: 'server', value: 1, fixed: true, physics:false});
    nodes.push({id: 1003, x: x, y: y + 3 * step, label: 'Computer', group: 'desktop', value: 1, fixed: true, physics:false});
    nodes.push({id: 1004, x: x, y: y + 4 * step, label: 'Smartphone', group: 'mobile', value: 1, fixed: true, physics:false});
}

```

```

    // create a network
    var container = document.getElementById('mynetwork');
    var data = {
        nodes: nodes \\数组[node 对象, node 对象]
        edges: edges \\数组[edge 对象,edge 对象]
    };
    var options = {
        nodes: { \\node 的全局属性
            scaling: {
                min: 16,
                max: 32
            }
        },
        edges: {
            color: GRAY,
            smooth: false
        },
        physics:{
            barnesHut:{gravitationalConstant:-30000},
            stabilization: {iterations:2500}
        },
        groups: {
            'switch': {
                shape: 'triangle',
                color: '#FF9900' // orange
            },
            desktop: {
                shape: 'dot',
                color: "#2B7CE9" // blue
            },
            mobile: {
                shape: 'dot',
                color: "#5A1E5C" // purple
            },
            server: {
                shape: 'square',
                color: "#C5000B" // red
            },
            internet: {
                shape: 'square',
                color: "#109618" // green
            }
        }
    };
    network = new vis.Network(container, data, options);
}
</script>
</head>

<body onload="draw()">                <body  onload="draw()">    libuv 的注册
    <div id="mynetwork"></div>
</body>
</html>

```

8 网络管理实践报告撰写大纲

1 网络实验实践完成情况与评分矩阵

网络管理课程实践评分标准与评分矩阵

网络管理课程实践评分标准与评分矩阵

实践内容	完成方法 A	分值	完成方法 B	分值	完成方法 C	分值
企业内部网络构建与 DNS 服务	企业内部网络构建	10	仅企业内部网络构建	10	仅企业内部网络构建	10
	DNS 服务工作正常	10	DNS 服务工作正常	10	DNS 服务工作正常	10
数据采集与显示	异步数据采集	20	同步数据采集	15	Solarwinds 数据采集与图形化显示 注: solarwinds 为基于 SNMP 协议的数据采集与显示客户端程序	20
	数据读取 list 显示	10	数据读取 list 显示	10		
	数据读取图形化显示	10	数据读取图形化显示	10		
拓扑结构监控	JS 编程实现拓扑结构监控	20	Js 编程实现静态拓扑结构	15	采用 what'sup 实现拓扑结构监控 注: what'sup 为拓扑结构监控客户端程序	10
实践报告	按格式完整编写实践报告	20	按格式要求编写实践报告	15	按格式完整撰写实践报告	10

2 企业内部网络构建

1.1 企业内部网络拓扑结构

1.2 核心交换机配置(含 AAA 认证)

1.2.1 C1 交换机配置

1.2.2 C2 交换机配置

1.2.3 D1 交换机配置

1.2.4 D2 交换机配置

1.2.5 R1 路由器配置

2.3 测试结果 (ping)

3 DNS 服务

3.1 named.conf 配置

3.2 客户端查询结果 (nslookup)

4 数据采集程序

4.1 数据采集程序流程图

4.2 源代码

4.3 工程文件(Makefile)

5 数据显示

5.1 数据 list 显示

5.1.1 数据 List 显示流程图

5.1.2 数据 list 显示源代码

5.2 数据图形化显示

5.2.1 数据图形化显示流程图

5.2.2 数据图表化显示源代码

6 拓扑结构监控

6.1 拓扑结构监控界面图

6.2 拓扑结构监控流程图

6.3 拓扑结构监控源代码

7 总结